

BookNest

Hotel Management & Booking System

Complete Technical Guide

& Demo Preparation Handbook

Every system in this project, explained from first principles — what it does, how it is wired, why we chose it, and the exact answers to the questions you will be asked.

Team Neuron

hms-and-booking.vercel.app

July 9, 2026

Contents

1	How To Use This Document	4
2	The Big Picture	5
2.1	What BookNest is	5
2.2	The technology, in one line each	5
2.3	The life of a request	5
3	Next.js: Structure and How Files Are Linked	7
3.1	The one rule that explains the whole folder	7
3.2	The special filenames	7
3.3	Server Components vs Client Components — the mental model	7
3.4	The four ways data moves in this app	8
3.4.1	1. Server Component reads the database directly	8
3.4.2	2. Route Handler + SWR (client-side fetching)	9
3.4.3	3. Server Actions	9
3.4.4	4. Realtime WebSocket subscription	10
3.5	Caching: how the public pages stay fast	10
3.6	proxy.ts: the gate in front of everything	10
3.7	The complete file map	11
4	Supabase: How the Database Is Implemented	13
4.1	What Supabase actually is	13
4.2	Why there are four Supabase clients	13
4.3	Row Level Security, properly explained	14
4.4	The RLS recursion problem (and how we solved it)	14
4.5	The trigger that creates every profile	15
4.6	Why so much logic lives in the database (RPC functions)	16
4.7	The complete table catalogue	16
4.8	Database functions, at a glance	17
4.9	Storage buckets	18
4.10	Performance: the indexes that matter	18
5	Authentication	19
5.1	The three sign-up paths	19
5.1.1	Guest, with email and password	19
5.1.2	Manager, with a verification document	19
5.1.3	Google OAuth	19
5.2	The callback route: one door, four destinations	20
5.3	getUser vs getClaims vs getSession	20
5.4	How the session survives	21
6	The Booking System	22
6.1	The model: room <i>types</i> , not room numbers	22
6.2	room_inventory: one row per room type per night	22
6.3	Checking availability	22
6.4	Creating the booking, atomically	23
6.4.1	Why FOR UPDATE is the whole ballgame	24
6.5	Why the price is calculated twice	24
6.6	Cancellation and the refund policy	24
6.7	Who is allowed to cancel	25

7	Payments: Razorpay	26
7.1	Why a gateway at all	26
7.2	The three-step handshake	26
7.3	Step 1: creating the order (server)	26
7.4	Step 2: the browser modal	27
7.5	Step 3: verifying the signature (server)	27
7.6	Three roads to “confirmed” — and why we need all three	28
7.7	Idempotency: sending the confirmation email exactly once	29
7.8	Razorpay Route: splitting the money	29
8	Real-Time Messaging	31
8.1	The data model	31
8.2	Unread counts are maintained by a trigger	31
8.3	Two kinds of realtime, used for two different things	31
8.4	Optimistic UI: why sending feels instant	32
8.5	Image compression, before upload	33
8.6	The privacy problem, and <code>shares_conversation_with</code>	33
8.7	The contact buttons themselves	34
9	QR Code Check-In	35
9.1	What the QR code actually contains	35
9.2	Scanning it (front desk)	35
9.3	From scanned text to a check-in	35
10	Maps	37
10.1	What we use, and what we deliberately did not	37
10.2	Vector tiles vs raster tiles	37
10.3	Where maps appear	37
11	Email Notifications	39
11.1	Why Brevo, and why HTTP instead of SMTP	39
11.2	The emails we send	39
11.3	Why email HTML is written with tables and inline styles	39
12	Managers, Staff and the Admin Panel	41
12.1	Staff are real accounts, not sub-users	41
12.2	The invite flow	41
12.3	Manager verification	41
12.4	Every admin action is an audited RPC	41
13	Every Package, and Why It Is There	43
13.1	Runtime dependencies	43
13.2	Development dependencies	44
13.3	Things we chose <i>not</i> to use	44
14	Security Summary	45
15	Frequently Asked Questions	46
15.1	Architecture and Next.js	46
15.2	Supabase and the database	47
15.3	Booking and concurrency	48
15.4	Payments	49
15.5	Messaging	50

15.6 QR check-in	51
15.7 Maps, email, and the rest	52
15.8 The hard questions	53
16 Known Limitations and Honest Notes	54
17 Appendix A: Environment Variables	55
18 Appendix B: Glossary	55

1. How To Use This Document

This document has three kinds of content, and you should read them differently.

1. **Explanations.** Plain-English descriptions of every system. Read these to *understand*. If a sentence uses a word you cannot define, the Glossary (Section 18) defines it.
2. **Code.** The actual code from the repository, with the file path above it. You should be able to point at any of these and explain what each line does.
3. **Green “SAY THIS” boxes.** A one-or-two-sentence answer, written the way you would speak it out loud. If you are asked a question and your mind goes blank, say the green box.

The single most important piece of advice. When someone asks “how does X work?”, do not start with the code. Start with the *problem* X solves, then say what you built, then offer the detail. For example: “*The problem is two people can book the last room at the same moment. So booking is not done in JavaScript — it is a Postgres function that takes a row lock on each night’s inventory before it writes. Want me to walk through it?*” That sentence alone tells an examiner you understand your own system.

2. The Big Picture

2.1 What BookNest is

BookNest is a hotel booking platform. It is not a toy: it takes real payments through a real payment gateway, sends real email, and enforces real security rules at the database level. It has four kinds of users:

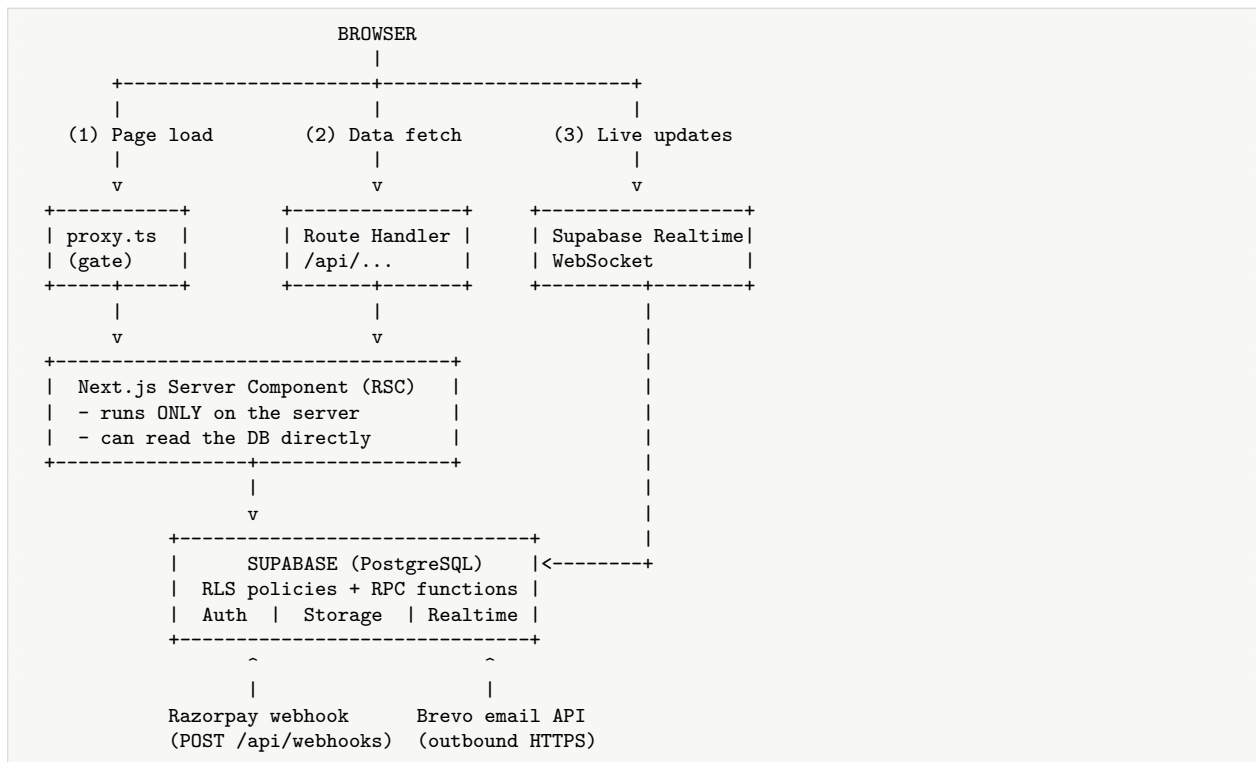
Role	What they can do
Guest	Search hotels, view listings, book rooms, pay online, check in with a QR code, review their stay, message the hotel.
Manager	Create hotel listings through a 9-step wizard, run a front desk (check guests in/out, take walk-in bookings), view earnings, invite staff, reply to guest messages.
Staff	A real login account invited by a manager. Gets a subset of the manager's powers on <i>one specific hotel</i> , controlled by a permissions array.
Admin	Approve or reject manager verification requests and hotel listings, suspend users, change roles, view platform statistics. Every admin action is written to an audit log.

2.2 The technology, in one line each

Layer	Choice and one-line justification
Framework	Next.js 16 (App Router). One codebase serves both the website and the API, so we never maintain two projects.
UI library	React 19 . Industry standard; Next.js is built on it.
Language	TypeScript . Catches type mistakes before the code runs.
Database	Supabase = hosted PostgreSQL + Auth + Storage + Realtime. One service instead of four.
Security model	Row Level Security (Postgres). The database itself refuses to return rows you are not allowed to see.
Payments	Razorpay . India-first, supports UPI, cards, netbanking. Has a proper test mode.
Realtime chat	Supabase Realtime (WebSockets over Postgres replication). No separate socket server to run.
Maps	MapLibre GL + OpenFreeMap , with Leaflet as a fallback. Zero API keys, zero cost, no billing surprises.
Email	Brevo HTTP API. Works from serverless functions (SMTP often does not) and has a free tier without needing a custom domain.
Styling	Tailwind CSS v4 . Styles live next to the markup.
Hosting	Vercel . Made by the Next.js team; push to GitHub and it deploys.

2.3 The life of a request

This diagram is worth memorising. Almost every question about “how does the app work” is answered by pointing at one box in it.



SAY THIS: *The browser talks to Next.js. Next.js talks to Postgres. But even if somebody bypassed Next.js entirely and hit the database directly with our public key, Row Level Security would still stop them from reading someone else's bookings. Security is enforced at the bottom of the stack, not the top.*

3. Next.js: Structure and How Files Are Linked

3.1 The one rule that explains the whole folder

In the Next.js **App Router**, a *folder is a URL*, and a file named *page.tsx* is what the user sees at that URL.

```
app/page.tsx           -> /
app/hotels/page.tsx    -> /hotels
app/hotels/[id]/page.tsx -> /hotels/<any-id>    ([id] = dynamic segment)
app/manager/staff/page.tsx -> /manager/staff
app/api/bookings/route.ts -> /api/bookings    (an API endpoint, not a page)
```

There is no router configuration file anywhere in this project. There is no `routes.js`. The folder structure *is* the routing table. This is why the `app/` directory looks the way it does.

3.2 The special filenames

Inside any folder, certain filenames have reserved meaning. These are the ones we use:

Filename	What Next.js does with it
<code>page.tsx</code>	The page itself. Must be present for the URL to exist.
<code>layout.tsx</code>	A wrapper that persists across navigations inside that folder. <code>app/layout.tsx</code> is the root layout (html/body, fonts, navbar). <code>app/manager/layout.tsx</code> wraps every manager page in the sidebar shell.
<code>loading.tsx</code>	Shown automatically while <code>page.tsx</code> is fetching data. Next.js wraps the page in a React <code><Suspense></code> boundary for you. This is why we have <code>app/hotels/loading.tsx</code> showing skeleton cards.
<code>template.tsx</code>	Like a layout, but re-created on every navigation. We use <code>app/manager/template.tsx</code> and <code>app/admin/template.tsx</code> for page transition animations, which need to re-mount to re-run.
<code>route.ts</code>	Turns the folder into an API endpoint instead of a page. Exports functions named <code>GET</code> , <code>POST</code> , etc.
<code>actions.ts</code>	Not a Next.js reserved name — <i>our</i> convention. Holds Server Actions (see below). Marked with <code>"use server"</code> at the top.
<code>proxy.ts</code>	Runs <i>before</i> every matching request. In Next.js 15 and earlier this file was called <code>middleware.ts</code> ; Next.js 16 renamed the convention to <code>proxy.ts</code> . Ours guards the protected routes.

Why is our middleware called `proxy.ts`? Because we are on Next.js 16, which renamed the convention. The function inside is still just a function that receives a request and returns a response. If an examiner says “I thought middleware was called `middleware.ts`”, they are right — for Next.js 15. This is a good thing to know; it shows you are on the current version and understand why.

3.3 Server Components vs Client Components — the mental model

This is the concept most people get wrong, so learn it properly.

By default, every component in `app/` is a Server Component. It runs on the server, once, and only its resulting HTML is sent to the browser. Its JavaScript is *never downloaded by the*

user. It can `await` directly. It can read the database. It can read secrets.

A file that starts with "use client" is a **Client Component**. Its JavaScript *is* shipped to the browser. It can use `useState`, `useEffect`, `onClick`, and browser APIs like the camera. It cannot read secrets, because the user can read its source.

```
export const revalidate = 60; // ISR: regenerate this page at most once a minute

export default async function HotelDetailPage({ params }) {
  const { id } = await params;

  // This runs on the SERVER. The user never sees this query.
  let detail = await getPublicHotel(id);
  if (!detail) notFound();

  return <div> ... </div>;
}
```

Listing 1: `/page.tsx` — a Server Component. Note: no "use client", and it awaits a database read directly.]`app/hotels/[id]/page.tsx` — a Server Component. Note: no "use client", and it awaits a database read directly.

The rule of thumb we followed across the whole codebase:

Fetch data in a Server Component. Pass it down as props into a small Client Component that handles the interactivity.

You can see this exact pattern everywhere:

- `app/dashboard/page.tsx` (server, fetches) renders `app/dashboard/DashboardClient.tsx` (client, interactive).
- `app/hotels/page.tsx` renders `app/hotels/HotelsClient.tsx`.
- `app/manager/staff/page.tsx` renders `StaffClient.tsx`.
- `app/messages/page.tsx` renders `GuestMessagesClient.tsx`.

SAY THIS: *Server Components render on the server and ship zero JavaScript, so the page is fast and the database query never reaches the browser. We only opt into a Client Component where we genuinely need interactivity — a map, a date picker, the chat box. That is why the naming convention across the repo is `page.tsx` for the server half and `SomethingClient.tsx` for the interactive half.*

3.4 The four ways data moves in this app

We do not use one data-fetching strategy. We deliberately use four, each where it fits. Being able to say *why* each one is used where it is used is the single most impressive thing you can do in this demo.

1. Server Component reads the database directly

Used for: public pages that must be fast and SEO-friendly (home page, hotel list, hotel detail page).

```
// app/hotels/[id]/page.tsx -- runs on server, no API route involved
const detail = await getPublicHotel(id); // lib/hotels.ts -> Supabase
```

No API endpoint exists for this. The page talks to Postgres and returns HTML. One network round trip for the user instead of two.

2. Route Handler + SWR (client-side fetching)

Used for: dashboards, where data changes and the user expects it to refresh.

```
export const fetcher = (url: string) => {
  fetch(url).then((r) => {
    if (!r.ok) throw new Error(`Request to ${url} failed with status ${r.status}`);
    return r.json();
  });
}

// Maps a manager nav route to the API endpoint that page reads, for preloading.
export function managerApiForRoute(href: string): string | null {
  if (href.startsWith("/manager/hotels")) return "/api/manager/hotels";
  if (href.startsWith("/manager/manage")) return "/api/manager/manage";
  if (href.startsWith("/manager/staff")) return "/api/manager/staff";
  return null;
}
```

Listing 2: lib/swr.ts — one shared fetcher so preloading and fetching use the same cache key

SWR stands for *stale-while-revalidate*. When you open a page, SWR instantly shows the last data it had (stale), then quietly re-fetches in the background and swaps in the fresh data. The dashboard feels instant.

The `managerApiForRoute` function powers a nice trick: when you *hover* over a sidebar link, we call SWR's `preload()` for the endpoint that page will need. By the time you click, the data has already arrived.

3. Server Actions

Used for: writes triggered by a form or a button, where we want to avoid writing an API endpoint at all.

A Server Action is a function marked "use server" that you can *import and call directly from a Client Component*. Next.js secretly turns the call into an HTTP POST. You never write the fetch, the URL, or the JSON.

```
"use server";

export async function markRead(conversationId: string) {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return;

  const { data: conversation } = await supabase
    .from("conversations")
    .select("guest_id")
    .eq("id", conversationId)
    .single();
  if (!conversation) return;

  const isGuest = conversation.guest_id === user.id;
  await supabase
    .from("conversations")
    .update(isGuest ? { guest_unread: 0 } : { host_unread: 0 })
    .eq("id", conversationId);

  revalidatePath('/messages'); // tell Next.js those pages are stale
  revalidatePath('/manager/messages');
}
```

Listing 3: app/messages/actions.ts

In the chat hook we simply write `await apiMarkRead(id)`. It looks like a local function call. It is actually a secure server round trip.

The trap. A Server Action is a public HTTP endpoint whether you like it or not. Anyone can call it with any arguments. That is why *every* action in this codebase re-checks `auth.getUser()` and relies on RLS — we never assume the caller is who the UI says they are.

4. Realtime WebSocket subscription

Used for: chat only. The browser opens a persistent connection to Supabase and Postgres pushes new rows to it. Covered fully in Section 8.

3.5 Caching: how the public pages stay fast

Three layers, all visible in the code.

(a) **unstable_cache with a tag.** The approved-hotels query is expensive (it joins rooms, reviews and photos). We cache its *result* in Next.js's data cache, under the tag "hotels".

```
export const HOTELS_CACHE_TAG = "hotels";

export const getApprovedHotelsCached = unstable_cache(
  async (): Promise<ExploreHotel[]> => {
    const supabase = createPublicClient();
    const { data, error } = await supabase
      .from("hotels")
      .select('id, name, location, ..., rooms (price), reviews (rating),
              hotel_photos (url, category, sort_order)')
      .eq("status", "approved")
      .order("created_at", { ascending: false });
    if (error) throw error;
    return data ?? [];
  },
  ["approved-hotels-explore"], // cache key
  { tags: [HOTELS_CACHE_TAG], revalidate: 3600 }, // tag + 1 hour fallback
);
```

Listing 4: lib/hotels.ts

(b) **Tag invalidation.** When an admin approves a hotel, or a manager edits one, we call `revalidateTag("hotels")` (see `app/actions/hotels.ts` and `app/api/clear-cache/route.ts`). Every cached query carrying that tag is thrown away instantly. So the cache is one hour *or until something actually changes*, whichever comes first.

(c) **ISR on the page itself.** `app/hotels/[id]/page.tsx` exports `revalidate = 60`. The rendered HTML is stored on Vercel's CDN and regenerated at most once a minute.

SAY THIS: *The hotel page is cached at two levels: the database query result is cached under a tag, and the rendered HTML is cached on the CDN for 60 seconds. When an admin approves a listing we call `revalidateTag`, which invalidates the query cache immediately rather than waiting for the timer. So the site is fast and correct.*

3.6 proxy.ts: the gate in front of everything

```
export async function proxy(request: NextRequest) {
  return updateSession(request);
}

export const config = {
  // Only run on the protected route groups. Public pages skip middleware
  // entirely -- no auth round trip. API routes authenticate themselves.
  matcher: ["/dashboard/:path*", "/manager/:path*", "/admin/:path*"],
};
```

Listing 5: proxy.ts

The `matcher` is a deliberate performance decision. Middleware runs before *every* matching request, so if we matched `/` and `/hotels` too, every anonymous visitor to the home page would pay for an auth check they do not need. We match only the three private areas.

The real logic lives in `lib/supabase/middleware.ts`:

```
const isGuestArea = path.startsWith("/dashboard");
const isManagerArea = path.startsWith("/manager");
const isAdminArea = path.startsWith("/admin");
const isProtected = isGuestArea || isManagerArea || isAdminArea;

// getClaims() verifies the JWT locally -- no round trip to the Auth server.
const { data: claims } = await supabase.auth.getClaims();
const userId = claims?.claims?.sub ?? null;

if (!userId) {
  if (isProtected) return redirect("/login?redirect=" + path);
  return response; // public page: cost nothing
}
if (!isProtected) return response;

// Role + (for managers) verification status, fetched IN PARALLEL.
const [{ data: profile }, { data: mv }] = await Promise.all([
  supabase.from("profiles").select("role, suspended").eq("id", userId).single(),
  isManagerArea
    ? supabase.from("manager_verifications").select("status")
      .eq("user_id", userId).order("created_at", { ascending: false })
      .limit(1).maybeSingle()
    : Promise.resolve({ data: null })
]);

if (profile?.suspended) return redirectTo("/", "?suspended=1");
if (isAdminArea && role !== "admin") return redirectTo("/");
if (isGuestArea && role !== "guest") return redirectTo("/");
if (isManagerArea && role !== "manager" && role !== "staff") return redirectTo("/");

// Managers (not staff) are gated behind admin approval.
if (isManagerArea && role === "manager") {
  const onWaiting = path.startsWith("/manager/waiting");
  if (mv?.status !== "approved" && !onWaiting) return redirectTo("/manager/waiting");
  if (mv?.status === "approved" && onWaiting) return redirectTo("/manager/hotels");
}
```

Listing 6: lib/supabase/middleware.ts — the access-control cascade

Three things to point out if asked:

1. `getClaims()` not `getUser()`. `getUser()` makes a network call to Supabase's auth server on *every request*. `getClaims()` verifies the JWT signature locally using the project's public signing key. Same guarantee, no round trip.
2. The two queries run inside `Promise.all`, not one after the other.
3. Suspension is checked before role, so a suspended admin is still locked out.

Middleware is a convenience, not the security boundary. If middleware were the only protection, someone could hit `/api/admin/users` directly and bypass it (`/api` is not in the matcher). That is fine, because every admin API route independently calls `getAdminContext()`, and RLS would refuse the query anyway. Middleware exists to give users a nice redirect, not to keep attackers out.

3.7 The complete file map

```

HMS-and-booking/
|
|-- proxy.ts ..... Auth/role gate. Runs before /dashboard, /manager, /admin
|-- next.config.ts ..... Image host allowlist
|-- app/
|   |-- layout.tsx ..... Root layout: fonts, AuthProvider, CurrencyProvider, Navbar
|   |-- page.tsx ..... Landing page
|   |-- globals.css ..... Tailwind + design tokens (brand colours)
|   |
|   |-- hotels/
|   |   |-- page.tsx ..... Server: fetches approved hotels (cached)
|   |   |-- HotelsClient.tsx . Client: filters, grid/map toggle
|   |   |-- ExploreMap.tsx ... Client: MapLibre pins
|   |   |-- loading.tsx ..... Skeleton shown while page.tsx fetches
|   |   |-- '[id]/
|   |   |   |-- page.tsx ..... Server: getPublicHotel(), ISR revalidate=60
|   |   |   |-- components/ .. GalleryModal, BookingWidget, ReviewsSection, ...
|   |   |   |-- book/page.tsx The booking flow
|   |
|   |-- bookings/[id]/
|   |   |-- page.tsx ..... Booking detail + generates the check-in QR (server-side)
|   |   |-- check-in/ ..... Guest-facing QR display
|   |   |-- 'review/ ..... Leave a review after checkout
|   |
|   |-- messages/ ..... Guest inbox      (+ actions.ts: server actions)
|   |-- dashboard/ ..... Guest bookings  (role=guest only)
|   |
|   |-- manager/
|   |   |-- layout.tsx ..... Sidebar shell (ManagerShell)
|   |   |-- create-hotel/ .... 9-step listing wizard (steps/Step1..Step9)
|   |   |-- manage/[hotelId]/ FrontDesk: QR scanner, check-in/out, walk-in bookings
|   |   |-- staff/ ..... Invite + permission management
|   |   |-- 'waiting/ ..... Shown until an admin approves the manager
|   |
|   |-- admin/ ..... Dashboard, verifications, listings, users
|   |
|   |-- auth/callback/route.ts OAuth + email-confirmation landing; routes by role
|   |
|   |-- api/ ..... <-- every server endpoint
|   |   |-- bookings/ ..... GET list, POST create
|   |   |-- availability/ .... GET rooms free for a date range
|   |   |-- payments/razorpay/{order,verify,reconcile}
|   |   |-- webhooks/razorpay/ Razorpay calls THIS (server-authoritative)
|   |   |-- manager/{hotels,manage,staff}
|   |   |-- admin/{overview,users,listings,verifications}
|   |   |-- 'messages/{attachment,unread-count,...}
|   |
|-- lib/
|   |-- supabase/
|   |   |-- client.ts ..... Browser client (anon key, cookies)
|   |   |-- server.ts ..... Server client (anon key, request cookies)
|   |   |-- public.ts ..... Cookie-free client (safe inside unstable_cache)
|   |   |-- admin.ts ..... SERVICE ROLE - bypasses RLS. Webhooks only.
|   |   |-- 'middleware.ts .... The role cascade used by proxy.ts
|   |   |-- booking.ts ..... Price math, refund policy, date helpers
|   |   |-- razorpay.ts ..... Server SDK instance (holds the secret)
|   |   |-- razorpayCheckout.ts .. Browser: order -> checkout modal -> verify
|   |   |-- reconcile.ts ..... Safety net: ask Razorpay "was this actually paid?"
|   |   |-- email.ts ..... Brevo API + branded HTML layout
|   |   |-- emails/*.ts ..... One file per transactional email
|   |   |-- hotels.ts ..... Cached hotel queries + listing assembly
|   |   |-- compression.ts ..... Canvas-based client-side image compression
|   |   |-- auth.ts / authServer.ts
|   |   |-- 'types.ts ..... Every shared TypeScript type
|   |
|-- components/ ..... Shared UI (Navbar, AuthProvider, DatePicker, ...)
|-- messaging/ ..... ChatThread, Composer, useConversationMessages
|-- manager/ ..... ManagerShell, Panel
|-- 'admin/ ..... AdminShell
|
|-- supabase/ ..... 16 .sql migration files -- the real source of truth

```

4. Supabase: How the Database Is Implemented

4.1 What Supabase actually is

Supabase is a **PostgreSQL** database with four services bolted on:

- **Auth** — user accounts, passwords, Google OAuth, JWTs.
- **PostgREST** — automatically turns every table into a REST API.
- **Storage** — an S3-like file store, with permissions.
- **Realtime** — streams Postgres changes over WebSockets.

The idea that makes Supabase different. Because the database is exposed directly to the browser through PostgREST, the browser holds a real database API key (the “anon key”). That sounds terrifying until you understand **Row Level Security**: the database is told, per table, exactly which rows each user may see. The anon key gets you *to* the database; RLS decides what you get *out of* it. The key is public by design.

4.2 Why there are four Supabase clients

This is a question you will almost certainly be asked, because it looks like duplication. It is not. Each client exists because of a different constraint.

File	Key	Why it exists
client.ts	anon	Runs in the browser. Reads and writes the auth cookie so the session survives a refresh.
server.ts	anon	Runs in Server Components, Route Handlers and Server Actions. Reads the <i>current request's</i> cookies via <code>cookies()</code> , so queries run as the logged-in user and RLS applies to them.
public.ts	anon	Stateless, <i>cookie-free</i> . Needed because <code>unstable_cache</code> forbids touching request-bound data like cookies — a cached value must not depend on who asked for it. RLS still applies (anon can only read approved hotels).
admin.ts	service role	Bypasses RLS entirely. Used in exactly one situation: the Razorpay webhook, where there is no logged-in user because Razorpay's server is the caller. The service-role key is server-only and never reaches the browser.

```
// Stateless, cookie-free Supabase client using the public anon key. RLS still
// applies (anon can only read approved hotels and other public rows). Safe to
// use inside 'unstable_cache', which must not touch request-bound data like
// cookies -- unlike the SSR 'server.ts' client.
export function createPublicClient() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { auth: { persistSession: false, autoRefreshToken: false } },
  );
}
```

Listing 7: lib/supabase/public.ts — read the comment; it is the whole justification

SAY THIS: *Four clients, four reasons. The browser one manages cookies. The server one reads the request’s cookies so RLS knows who you are. The public one is cookie-free because Next.js caching forbids request-scoped data inside a cached function. And the admin one uses the service-role key to bypass RLS — we use it in exactly one place, the Razorpay webhook, because there is no user session when a payment gateway calls us.*

4.3 Row Level Security, properly explained

Normally, an application checks permissions: “is this user allowed to see this booking? ok, run the query.” If you forget that check in one endpoint, the data leaks.

RLS inverts it. You attach a rule to the *table*, and Postgres appends it to every single query, forever, no matter who is asking or from where.

```
alter table public.bookings enable row level security;

create policy "bookings: read own"
  on public.bookings for select using (auth.uid() = guest_id);

create policy "bookings: read manager"
  on public.bookings for select
  using (exists (
    select 1 from public.hotels h
    where h.id = hotel_id and h.manager_id = auth.uid()
  ));

create policy "bookings: read admin"
  on public.bookings for select using (public.get_my_role() = 'admin');
```

Listing 8: supabase/booking_schema.sql

Now `select * from bookings` returns: your own bookings if you are a guest; your hotels’ bookings if you are a manager; everything if you are an admin; and **nothing** if you are a stranger — even with a valid API key. Multiple `SELECT` policies are OR-ed together.

`auth.uid()` is a Postgres function Supabase provides. It reads the user’s ID out of the JWT attached to the request.

4.4 The RLS recursion problem (and how we solved it)

This is a genuinely hard problem we hit, and explaining it will impress anyone.

Consider: the policy on `hotels` needs to check “is the caller assigned staff on this hotel?”, which requires reading `hotel_staff`. But the policy on `hotel_staff` needs to check “does the caller own this hotel?”, which requires reading `hotels`. Each table’s policy queries the other table — whose policy queries the first table. Postgres detects the loop and throws `infinite recursion detected in policy`.

The fix is `SECURITY DEFINER`. A function declared `SECURITY DEFINER` runs with the privileges of the user who *created* it (the database owner), not the user calling it — so RLS does not apply to the queries *inside* it. It reads one table, cleanly, and returns a boolean.

```
-- Single-table definer helpers used inside RLS policies to AVOID the mutual
-- recursion hotels <-> hotel_staff. These bypass RLS internally.

create or replace function public.owns_hotel(p_hotel_id uuid)
returns boolean language sql stable
security definer set search_path = public
as $$ select exists (select 1 from public.hotels h
                    where h.id = p_hotel_id and h.manager_id = auth.uid()); $$;

-- True if the caller is the hotel’s manager OR staff holding ‘p_perm’.
```



```

create or replace function public.can_manage_hotel(p_hotel_id uuid, p_perm text)
returns boolean language sql stable
security definer set search_path = public
as $$
select
  exists (select 1 from public.hotels h
          where h.id = p_hotel_id and h.manager_id = auth.uid())
or exists (select 1 from public.hotel_staff s
          where s.hotel_id = p_hotel_id and s.staff_id = auth.uid()
          and p_perm = any (s.permissions));
$$;

```

Listing 9: supabase/hms_management_migration.sql

Why set search_path = public? A SECURITY DEFINER function runs as a privileged user. If an attacker could change the schema search path, they could make the function call *their* malicious `public.hotels` instead of ours. Pinning `search_path` closes that hole. Every definer function in this project pins it. This is the single most commonly forgotten line in Postgres security, and we did not forget it.

The same technique is used in messaging: `can_access_conversation()` and `validate_message_sender()` exist purely so the `messages` policies never have to query `conversations` under RLS.

4.5 The trigger that creates every profile

Supabase stores users in `auth.users`, a schema we do not control. We need our own `public.profiles` row for each of them (holding the role). We do this with a trigger, so it is impossible for a user to exist without a profile.

```

create or replace function public.handle_new_user()
returns trigger language plpgsql
security definer set search_path = public
as $$
begin
  insert into public.profiles (id, role, full_name, phone, dob, location)
  values (
    new.id,
    coalesce(new.raw_user_meta_data ->> 'role', 'guest'), -- default: guest
    new.raw_user_meta_data ->> 'full_name',
    new.raw_user_meta_data ->> 'phone',
    nullif(new.raw_user_meta_data ->> 'dob', '')::date,
    new.raw_user_meta_data ->> 'location'
  )
  on conflict (id) do nothing;

  -- Managers additionally get a pending verification row, automatically.
  if coalesce(new.raw_user_meta_data ->> 'role', 'guest') = 'manager' then
    insert into public.manager_verifications (
      user_id, business_name, registration_number, business_address,
      document_url, status)
    values (new.id,
      new.raw_user_meta_data ->> 'business_name',
      new.raw_user_meta_data ->> 'registration_number',
      new.raw_user_meta_data ->> 'business_address',
      new.raw_user_meta_data ->> 'document_url',
      'pending');
  end if;
  return new;
end; $$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_user();

```

Listing 10: supabase/schema.sql

`raw_user_meta_data` is the JSON blob we passed in the `options.data` of `supabase.auth.signUp()`. So the client says “I am signing up as a manager, here is my business name”, and the database turns that into a profile *plus* a pending verification request — in one transaction, before the user has even confirmed their email.

SAY THIS: *We never insert a profile from application code. A Postgres trigger on `auth.users` does it, reading the sign-up metadata. That means a user row without a matching profile row is structurally impossible — there is no code path that can forget.*

4.6 Why so much logic lives in the database (RPC functions)

A large amount of this project’s business logic is written in PL/pgSQL and called via `supabase.rpc("function_name", {...})`. That is a deliberate architectural choice, and here is the reasoning.

1. **Atomicity.** `book_room` reserves inventory for every night *and* writes the booking *and* writes the payment row. If any step fails, Postgres rolls the whole thing back. Doing this from JavaScript across three separate HTTP calls would leave orphaned data the moment the network hiccups.
2. **Concurrency.** Only the database can take a row lock. See Section 6.4.
3. **Trust.** The price is computed in SQL from the room’s real price. A malicious client that POSTs `{"total": 1}` is ignored — the server never reads a price from the request body.
4. **One place to change.** Both the web app and the webhook confirm payments. They both call the same function.

4.7 The complete table catalogue

Table	Purpose and key columns
<code>profiles</code>	One row per auth user. Holds <code>role</code> (guest / manager / staff / admin), <code>name</code> , <code>phone</code> , <code>suspended</code> .
<code>manager_verifications</code>	A manager’s business documents and approval <code>status</code> (pending / approved / rejected / more_info).
<code>hotels</code>	The listing. <code>status</code> gates visibility (draft → pending → approved). Also holds <code>lat/lng</code> , <code>amenities</code> array, <code>policies</code> , <code>gst_percent</code> , <code>deleted_at</code> (soft delete).
<code>rooms</code>	A room <i>type</i> (not a physical room). <code>price</code> , <code>capacity</code> , <code>total_units</code> = how many of this type exist.
<code>room_inventory</code>	The heart of availability. One row per (room type, date), holding <code>total_count</code> and <code>booked_count</code> , with a CHECK constraint that booked never exceeds total.
<code>bookings</code>	<code>status</code> in pending / confirmed / checked_in / completed / cancelled. Snapshots the price at booking time. <code>guest_id</code> is nullable to support walk-in bookings.
<code>payments</code>	One per booking. <code>order_id</code> (Razorpay order), <code>transaction_id</code> (Razorpay payment id), <code>status</code> .
<code>conversations</code>	One per (hotel, guest) pair — enforced by a UNIQUE constraint. Holds <code>guest_unread</code> , <code>host_unread</code> , and a preview of the last message.
<code>messages</code>	<code>body</code> , <code>attachments</code> (jsonb array), <code>sender_role</code> .

Table	Purpose and key columns
hotel_staff	Assignment of a staff account to one hotel, with a <code>permissions text[]</code> array.
staff_invites	Pending email invitations, with a random <code>token</code> and a 14-day <code>expires_at</code> .
reviews	rating 1–5 + comment.
hotel_photos, room_photos	Ordered image URLs.
admin_audit_log	Every admin action, with a jsonb <code>details</code> blob. Append-only; admins can read but the app never deletes.

4.8 Database functions, at a glance

Function	What it does
<code>get_my_role()</code>	Returns the caller's role without tripping RLS recursion on <code>profiles</code> . Used inside dozens of policies.
<code>handle_new_user()</code>	Trigger. Creates the profile on signup.
<code>room_availability(hotel, in, out)</code>	Returns each room type with the number of units free for the <i>whole</i> range.
<code>book_room(...)</code>	Atomically reserves inventory, creates the booking and the pending payment. Returns the booking id.
<code>book_offline(...)</code>	Same, for a walk-in guest with no account.
<code>attach_payment_order(booking_order)</code>	Stores the Razorpay order id.
<code>confirm_razorpay_payment(...)</code>	Marks payment completed + booking confirmed. Only after the server has verified the signature.
<code>cancel_booking(booking, reason)</code>	Releases inventory night by night and computes the refund from the time-based policy.
<code>check_in_booking(id)</code> / <code>check_out_booking(id)</code>	Front-desk state transitions, guarded by <code>can_operate_hotel()</code> .
<code>can_manage_hotel(hotel, perm)</code>	Manager OR staff-with-permission.
<code>can_operate_hotel(hotel)</code>	Manager OR any assigned staff.
<code>shares_conversation_with(profile)</code>	Enforces the privacy rule that lets a host read a guest's profile only when a conversation exists.
<code>get_hotel_reviews(hotel)</code>	Joins reviews to profiles internally and returns <i>only</i> safe fields (name, join date) — never phone or email.
<code>admin_review_hotel(...)</code> , <code>admin_review_manager(...)</code> , <code>admin_set_user_role(...)</code> , <code>admin_set_user_suspended(...)</code>	Every one begins with <code>if get_my_role() <> 'admin' then raise exception</code> and ends by writing to <code>admin_audit_log</code> .

4.9 Storage buckets

Bucket	Public?	Policy
manager-documents	No	Insert allowed anonymously (the upload happens <i>during</i> signup, before a session exists); read restricted to admins.
hotel-images	Yes	Anyone reads; authenticated users write.
message-attachments	No	Made private in a later migration. Reads go through a signed-URL endpoint that checks conversation membership.

4.10 Performance: the indexes that matter

```
-- Trigram matching lets 'ILIKE '%term%' (leading-wildcard) use an index
-- instead of scanning every row. Required for instant text search.
create extension if not exists pg_trgm;

create index if not exists hotels_location_trgm_idx
  on public.hotels using gin (location gin_trgm_ops);
create index if not exists hotels_name_trgm_idx
  on public.hotels using gin (name gin_trgm_ops);

-- The catalog query is: status = 'approved' ORDER BY created_at DESC.
-- A partial index over ONLY approved rows serves the filter and the sort
-- with no extra sort step.
create index if not exists hotels_approved_created_idx
  on public.hotels (created_at desc)
  where status = 'approved';
```

Listing 11: supabase/perf_migration.sql

SAY THIS: Searching for “goa” means *ILIKE '%goa%'*, and a normal B-tree index cannot help with a leading wildcard — Postgres would scan every hotel. So we enabled the *pg_trgm* extension and built GIN trigram indexes, which index every three-character substring. And because the catalogue query always filters on *status = 'approved'*, we used a partial index — it only contains approved rows, so it is smaller and it satisfies the sort as well as the filter.

5. Authentication

5.1 The three sign-up paths

Guest, with email and password

```
export async function signUpGuest(input: GuestSignupInput) {
  const supabase = createClient();
  const { data, error } = await supabase.auth.signUp({
    email: input.email,
    password: input.password,
    options: {
      emailRedirectTo: `${siteUrl}/auth/callback`,
      data: { // -> becomes raw_user_meta_data
        role: "guest",
        full_name: input.fullName,
        dob: input.dob,
        phone: input.phone,
        location: input.location,
      },
    },
  });
  if (error) throw error;
  return data;
}
```

Listing 12: lib/auth.ts

We never write to profiles here. We hand the data to Supabase Auth, and the `handle_new_user` trigger creates the profile. This works even before the user confirms their email.

Manager, with a verification document

The document must be uploaded *before* `signUp()` returns, because the trigger needs its path in the metadata. This is why the `manager-documents` bucket permits anonymous inserts.

```
const path = `pending/${crypto.randomUUID()}.${ext}`;
await supabase.storage.from("manager-documents").upload(path, input.document);

await supabase.auth.signUp({
  email: input.email, password: input.password,
  options: { data: { role: "manager", business_name: ..., document_url: path } },
});
```

Listing 13: lib/auth.ts

Google OAuth

```
export async function signInWithGoogle() {
  const supabase = createClient();
  // Always runs in the browser (click handler), so prefer the live origin.
  // This keeps localhost, Vercel previews, and production each redirecting
  // back to themselves rather than a single hard-coded site URL.
  const origin = typeof window !== "undefined" ? window.location.origin : siteUrl;
  return supabase.auth.signInWithOAuth({
    provider: "google",
    options: { redirectTo: `${origin}/auth/callback` },
  });
}
```

Listing 14: lib/auth.ts — note the origin trick

Google only gives us an email and a name. It gives us no phone number and no date of birth. So the trigger creates a profile with those fields null, and `app/auth/callback/route.ts` notices and

sends the user to `/onboarding` to fill them in.

5.2 The callback route: one door, four destinations

```
export async function GET(request: Request) {
  const code = searchParams.get("code");
  if (!code) return NextResponse.redirect(`${origin}/login?error=missing_code`);

  const supabase = await createClient();
  const { error } = await supabase.auth.exchangeCodeForSession(code);
  if (error) return NextResponse.redirect(`${origin}/login?error=auth_failed`);

  const { data: profile } = await supabase
    .from("profiles").select("role, phone").eq("id", user.id).single();
  const role = profile?.role ?? "guest";

  if (role === "admin") return redirect(`${origin}/admin/dashboard`);

  if (role === "manager") {
    // approved? -> dashboard
    // else -> waiting room
    const { data: mv } = await supabase
      .from("manager_verifications").select("status")
      .eq("user_id", user.id).order("created_at", { ascending: false })
      .limit(1).maybeSingle();
    return redirect(`${origin}${mv?.status === "approved"
      ? "/manager/dashboard" : "/manager/waiting"}`);
  }

  // Guest. OAuth sign-ups land here with no phone -> finish the profile first.
  if (!profile?.phone) return redirect(`${origin}/onboarding`);
  return redirect(`${origin}/`);
}
```

Listing 15: `app/auth/callback/route.ts`

Both the email-confirmation link and the Google redirect land here. The code is a one-time authorisation code; `exchangeCodeForSession` swaps it for a session cookie. This is the PKCE flow, and Supabase implements it for us.

5.3 `getUser` vs `getClaims` vs `getSession`

We use all three, in different places, on purpose. If you can explain this, you understand the auth system better than most working developers.

Call	Cost	Where we use it, and why
<code>getSession()</code>	Free (reads local storage)	<code>components/AuthProvider.tsx</code> . It is only driving UI state (“show the avatar”). It can be trusted for cosmetics, never for authorisation.
<code>getClaims()</code>	Cheap (verifies the JWT signature locally)	<code>lib/supabase/middleware.ts</code> and <code>lib/authServer.ts</code> . We only need the user id, and a forged token fails signature verification.
<code>getUser()</code>	Expensive (network call to Supabase Auth)	Route Handlers that write — <code>api/bookings</code> , <code>api/payments/*</code> . Here we want the strongest possible guarantee before touching money.

```
// getSession() reads the token from local storage (instant, no network
// round-trip), unlike getUser() which validates against Supabase's servers
// on every page. For client-side UI state this is the recommended call;
// anything security-sensitive is enforced server-side / by RLS anyway.
```

```
const { data: { session } } = await supabase.auth.getSession();
```

Listing 16: components/AuthProvider.tsx — the comment explains the trade-off

SAY THIS: *getSession* just reads the token from local storage — fast, but a user could edit local storage, so we only use it to decide what to draw. *getClaims* cryptographically verifies the token's signature locally, which is enough to trust the user id without a network call — that is what middleware uses. *getUser* actually asks Supabase's server, and we save it for the routes that create bookings and payments.

5.4 How the session survives

1. Supabase issues a short-lived JWT (access token) and a refresh token.
2. `@supabase/ssr` stores them in **cookies**, not just local storage, so the *server* can read them too.
3. On a protected route, `proxy.ts` runs, and the act of constructing the server client refreshes the token if needed and writes the new cookie onto the response.
4. On public routes we skip middleware entirely — the browser client refreshes its own token in the background.

That fourth point is why the `matcher` in `proxy.ts` is safe to restrict.

6. The Booking System

6.1 The model: room *types*, not room numbers

We do not store “Room 302”. We store a room *type* — “Deluxe Suite” — with a `total_units` count of how many exist. Availability is therefore a counting problem, not an assignment problem.

Why? Because guests do not book Room 302; they book “a deluxe room”. The hotel assigns the physical room at check-in. Modelling it as a count is both simpler and closer to how hotels actually operate. It also means the front desk does not have to maintain a floor plan in our system.

6.2 `room_inventory`: one row per room type per night

```
create table public.room_inventory (
  id          uuid primary key default gen_random_uuid(),
  room_id     uuid not null references public.rooms (id) on delete cascade,
  date        date not null,
  total_count int not null,
  booked_count int not null default 0,
  unique (room_id, date),
  constraint inventory_not_overbooked check (booked_count <= total_count)
);
```

Two details worth pointing at:

- `unique (room_id, date)` — there can only ever be one inventory row per night. This is what makes the lock in `book_room` work.
- `check (booked_count <= total_count)` — a database-level guarantee that overbooking is *impossible*, even if every line of application code were wrong.

Rows are created **lazily**. A hotel with 20 room types open for two years would otherwise need 14,600 rows on day one. Instead the row for a given night springs into existence the first time somebody books that night.

6.3 Checking availability

```
-- A room is bookable for the whole range only if it has stock on EVERY night,
-- so availability = total_units - max(booked nights in range).
create or replace function public.room_availability(
  p_hotel_id uuid, p_check_in date, p_check_out date)
returns table (room_id uuid, name text, price numeric,
              capacity int, amenities text[], available int)
language sql stable security definer set search_path = public
as $$
  select r.id, r.name, r.price, r.capacity, r.amenities,
         r.total_units - coalesce((
           select max(i.booked_count)
           from public.room_inventory i
           where i.room_id = r.id
             and i.date >= p_check_in
             and i.date < p_check_out      -- note: check-out night is NOT occupied
         ), 0) as available
  from public.rooms r
  where r.hotel_id = p_hotel_id
  order by r.price asc;
$$;
```

Listing 17: supabase/booking_schema.sql

The clever bit is `max(booked_count)`. If a guest wants three nights and the room is fully booked

on the middle night, they cannot have the range at all. Taking the maximum booked count across the range gives exactly the number of units free for the *entire* stay.

`i.date < p_check_out` (not `<=`) encodes the fact that you do not occupy a room on the night of your check-out day.

6.4 Creating the booking, atomically

This is the most important 30 lines in the project. Learn them.

```
create or replace function public.book_room(
    p_room_id uuid, p_check_in date, p_check_out date,
    p_guests int, p_num_rooms int, p_special text)
returns uuid language plpgsql security definer set search_path = public
as $$
declare v_room public.rooms; v_hotel public.hotels;
        v_uid uuid := auth.uid(); v_nights int; v_avail int; d date;
        v_base numeric; v_gst numeric; v_service_charge numeric;
        v_fee numeric; v_total numeric; v_id uuid;
begin
    if v_uid is null then raise exception 'Not authenticated'; end if;
    if p_check_out <= p_check_in then raise exception 'Check-out must be after check-in'; end if;
    if p_check_in < current_date then raise exception 'Check-in cannot be in the past'; end if;
    if p_num_rooms < 1 then raise exception 'At least one room is required'; end if;

    select * into v_room from public.rooms where id = p_room_id;
    select * into v_hotel from public.hotels where id = v_room.hotel_id;
    v_nights := p_check_out - p_check_in;

    -- Reserve each night: lazily create the inventory row, LOCK it, then book.
    d := p_check_in;
    while d < p_check_out loop
        insert into public.room_inventory (room_id, date, total_count, booked_count)
            values (p_room_id, d, v_room.total_units, 0)
            on conflict (room_id, date) do nothing;           -- (1) idempotent create

        select total_count - booked_count into v_avail
            from public.room_inventory
            where room_id = p_room_id and date = d
            for update;                                       -- (2) THE ROW LOCK

        if v_avail < p_num_rooms then
            raise exception 'Only % room(s) left on %', v_avail, to_char(d, 'Mon DD');
        end if;                                              -- (3) rolls back everything

        update public.room_inventory
            set booked_count = booked_count + p_num_rooms
            where room_id = p_room_id and date = d;

        d := d + 1;
    end loop;

    -- Price is computed HERE, from the DB's own numbers. Never from the client.
    v_base      := v_room.price * p_num_rooms * v_nights;
    v_gst        := round(v_base * (coalesce(v_hotel.gst_percent, 18.00) / 100.0), 2);
    v_service_charge := round(v_base * (coalesce(v_hotel.service_charge_percent, 0) / 100.0), 2);
    v_fee        := round(v_base * 0.02, 2);                -- platform fee
    v_total      := v_base + v_gst + v_service_charge + v_fee;

    insert into public.bookings (...) values (... , 'pending', p_special)
        returning id into v_id;
    insert into public.payments (booking_id, amount, payment_method, status)
        values (v_id, v_total, 'upi', 'pending');

    return v_id;
end; $$;
```

Listing 18: supabase/booking_schema.sql — book_room

Why FOR UPDATE is the whole ballgame

Imagine one room left, and two guests click “Book” in the same millisecond.

```
WITHOUT the lock:
Alice reads booked_count = 4, total = 5 -> "1 free, ok!"
Bob   reads booked_count = 4, total = 5 -> "1 free, ok!"
Alice writes booked_count = 5
Bob   writes booked_count = 5          <-- Bob overwrote Alice. OVERBOOKED.

WITH 'for update':
Alice's SELECT ... FOR UPDATE locks the row for date D.
Bob's SELECT ... FOR UPDATE *blocks* -- it waits.
Alice writes booked_count = 5, her transaction commits, lock released.
Bob's SELECT now returns booked_count = 5 -> v_avail = 0
Bob raises 'Only 0 room(s) left on Mar 14' -> his whole transaction ROLLBACKS.
```

And because the entire function is one transaction, when Bob’s exception fires on night three, the reservations he already made for nights one and two are undone automatically. There is no cleanup code, and there cannot be a partial booking.

SAY THIS: *Two people booking the last room at the same instant is a classic race condition. We solve it the way a bank does: `SELECT ... FOR UPDATE` takes a row-level lock on each night’s inventory row, so the second transaction physically waits for the first to commit and then sees the updated count. The whole function is one transaction, so if night three is full, nights one and two are rolled back automatically. And there is a `CHECK` constraint as a last line of defence — even a bug could not overbook the hotel.*

6.5 Why the price is calculated twice

`lib/booking.ts` has `computeQuote()`, which mirrors the SQL exactly. This is not duplication by accident:

```
export const GST_RATE = 0.18;
export const PLATFORM_FEE_RATE = 0.02;

// Mirror of the server-side price math in book_room (kept in sync for preview).
export function computeQuote(roomPrice, nights, numRooms,
                             gstPercent = 18, serviceChargePercent = 0): PriceQuote {
  const base      = round2(roomPrice * nights * numRooms);
  const gst       = round2(base * (gstPercent / 100));
  const serviceCharge = round2(base * (serviceChargePercent / 100));
  const platformFee = round2(base * PLATFORM_FEE_RATE);
  const total = round2(base + gst + serviceCharge + platformFee);
  return { roomPrice, nights, numRooms, base, gst, serviceCharge, platformFee, total };
}
```

Listing 19: `lib/booking.ts`

The TypeScript version exists *only* to render the price breakdown while the user is choosing dates, with no network call. The number it produces is never sent to the server and never trusted. `book_room` recomputes from `rooms.price` and `hotels.gst_percent`.

SAY THIS: *The client-side calculation is a preview, for instant feedback. The authoritative number comes from `book_room`, which reads the price out of the database. If you tampered with the JavaScript to say the room costs one rupee, the booking would still be created for the real amount — the API never accepts a price as input.*

6.6 Cancellation and the refund policy

```

v_hours := extract(epoch from (v_b.check_in::timestamp - now())) / 3600;
if v_hours >= 48 then v_pct := 1.0; -- full refund
elsif v_hours >= 24 then v_pct := 0.5; -- half
else v_pct := 0.0; -- none
end if;
v_refund := round(v_b.total_price * v_pct, 2);

-- Release the reserved inventory, night by night.
d := v_b.check_in;
while d < v_b.check_out loop
  update public.room_inventory
    set booked_count = greatest(0, booked_count - v_b.num_rooms)
    where room_id = v_b.room_id and date = d;
  d := d + 1;
end loop;

```

Listing 20: supabase/booking_schema.sql — cancel_booking

`greatest(0, ...)` is defensive: even if something else had already decremented the count, we can never drive it negative.

There is a mirror of this policy in TypeScript too (`refundPolicy()` in `lib/booking.ts`) — again, display only. The comment in the source says so explicitly: “*Refund policy mirror (display only — server recomputes authoritatively).*”

6.7 Who is allowed to cancel

```

if v_b.guest_id <> auth.uid() and public.get_my_role() <> 'admin' then
  raise exception 'Not allowed';
end if;

```

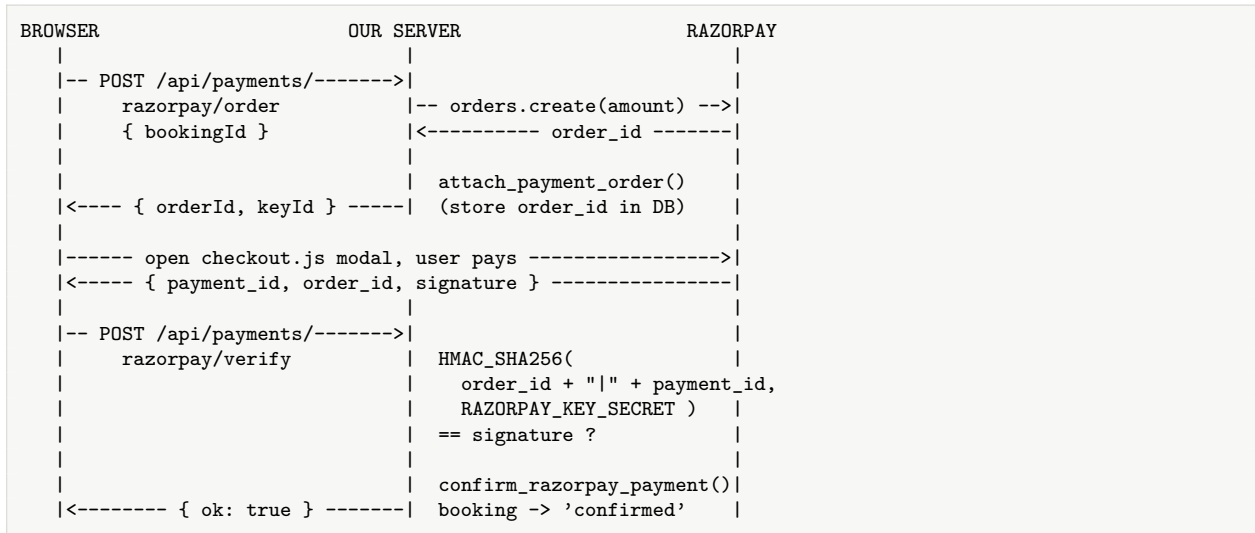
Later, `hms_management_migration.sql` *redefines* the same function to also permit the hotel’s manager and staff, so the front desk can cancel a booking for a guest who calls the hotel. This is why the same function name appears in two migration files — the later one wins. That is intentional, and it is how migrations work: they are applied in order, and each is idempotent (`create or replace, if not exists`).

7. Payments: Razorpay

7.1 Why a gateway at all

We never see a card number. If we did, we would be legally responsible for PCI-DSS compliance. Razorpay's checkout runs on *Razorpay's* domain, inside a modal. The card details go from the user's browser straight to Razorpay. Our server only ever handles opaque identifiers.

7.2 The three-step handshake



7.3 Step 1: creating the order (server)

```
const { data: { user } } = await supabase.auth.getUser();
if (!user) return NextResponse.json({ error: "Not authenticated" }, { status: 401 });

// RLS ensures the booking belongs to this user.
const { data: booking } = await supabase
  .from("bookings")
  .select("total_price, base_price, status, hotel_id, hotels (require_advance, ...)")
  .eq("id", bookingId).maybeSingle();

if (!booking) return NextResponse.json({ error: "Booking not found" }, { status: 404 });
if (booking.status !== "pending")
  return NextResponse.json({ error: "This booking is not awaiting payment" }, { status: 400 });

// Prices are stored in INR, so the order amount is simply paise.
const amountPaise = Math.round(amountToCharge * 100);

const order = await razorpay.orders.create({
  amount: amountPaise, currency: "INR",
  receipt: bookingId, notes: { bookingId },
  ...(transfers ? { transfers } : {}), // Razorpay Route split, see below
});

await supabase.rpc("attach_payment_order", {
  p_booking_id: bookingId, p_order_id: order.id,
});

return NextResponse.json({
  orderId: order.id, amount: amountPaise, currency: "INR",
  keyId: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID, // publishable, safe
});
```

Listing 21: app/api/payments/razorpay/order/route.ts (abridged)

Three deliberate details:

- The amount comes from `booking.total_price` in the database. The request body contains *only* a `bookingId`.
- “Booking not found” is what an attacker sees when they pass someone else’s booking id — because RLS returned zero rows. We did not have to write an ownership check; the database performed it.
- Razorpay works in **paise**, the smallest currency unit. Rs. 1,500 is 150000. Using integers avoids floating-point money bugs.

7.4 Step 2: the browser modal

```
const ok = await loadScript("https://checkout.razorpay.com/v1/checkout.js");

const rzp = new window.Razorpay({
  key: order.keyId, order_id: order.orderId,
  amount: order.amount, currency: order.currency,
  name: "BookNest",
  handler: async (resp: RazorpayResponse) => { // Razorpay calls this on success
    const v = await fetch("/api/payments/razorpay/verify", {
      method: "POST",
      body: JSON.stringify({
        bookingId,
        orderId: resp.razorpay_order_id,
        paymentId: resp.razorpay_payment_id,
        signature: resp.razorpay_signature,
      }),
    });
    if (v.ok) onSuccess(); else onError(...);
  },
  modal: { ondismiss: () => onError("Payment was cancelled.") },
});
rzp.open();
```

Listing 22: `lib/razorpayCheckout.ts`

7.5 Step 3: verifying the signature (server)

```
// Signature = HMAC_SHA256(order_id + "|" + payment_id, RAZORPAY_KEY_SECRET)
const expected = crypto
  .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET!)
  .update(`${orderId}|${paymentId}`)
  .digest("hex");

// Timing-safe comparison.
const valid =
  expected.length === signature.length &&
  crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(signature));

if (!valid)
  return NextResponse.json({ error: "Payment signature verification failed" }, { status: 400 });

const { error } = await supabase.rpc("confirm_razorpay_payment", {
  p_booking_id: bookingId, p_order_id: orderId, p_payment_id: paymentId,
});
```

Listing 23: `app/api/payments/razorpay/verify/route.ts`

What is an HMAC, in plain English? It is a fingerprint of a message, made using a secret only two parties know. Razorpay knows our `KEY_SECRET`. We know our `KEY_SECRET`. Nobody else does. So when Razorpay sends back a fingerprint of `order_id|payment_id` and we compute the same fingerprint and it matches, we know the message really came from

Razorpay and was not altered. A hacker cannot forge it without the secret.

Why `timingSafeEqual` and not `===`? Because `===` on strings stops comparing at the first mismatched character. An attacker who can measure response times very precisely could learn the signature one character at a time — a *timing attack*. `crypto.timingSafeEqual` always compares every byte, so the comparison takes constant time regardless of where the strings differ.

SAY THIS: *Anyone can POST to our verify endpoint with a made-up payment id. The signature is what makes it trustworthy: it is an HMAC-SHA256 of the order id and payment id, keyed with a secret that only Razorpay and our server hold. We recompute it and compare in constant time. If it does not match, nothing happens.*

7.6 Three roads to “confirmed” — and why we need all three

A payment can succeed while the user’s Wi-Fi dies, or their phone battery goes flat, or they close the tab. The money left their account. Our database still says `pending`. That is unacceptable, so there are three independent paths:

Path	Trigger	Mechanism
Happy	Razorpay’s JS calls our handler	<code>api/payments/razorpay/verify</code> verifies the signature and confirms.
Webhook	Razorpay’s <i>server</i> POSTs to us	<code>api/webhooks/razorpay</code> verifies an HMAC of the raw request body against <code>RAZORPAY_WEBHOOK_SECRET</code> , then confirms using the service-role client (there is no user session).
Reconcile	The user re-opens the booking page	<code>lib/reconcile.ts</code> asks Razorpay “was order X actually paid?” via <code>orders.fetchPayments()</code> and confirms if a captured payment exists.

```
export async function reconcilePendingBooking(supabase, bookingId, orderId) {
  if (!orderId) return false;
  try {
    const res = await razorpay.orders.fetchPayments(orderId);
    const captured = res.items?.find((p) => p.status === "captured");
    if (!captured) return false;

    const { error } = await supabase.rpc("confirm_razorpay_payment", {
      p_booking_id: bookingId, p_order_id: orderId, p_payment_id: captured.id,
    });
    return !error;
  } catch { return false; }
}
```

Listing 24: `lib/reconcile.ts` — the webhook-free safety net

```
const signature = request.headers.get("x-razorpay-signature") ?? "";
const raw = await request.text(); // must be the raw text, not parsed JSON

const expected = crypto.createHmac("sha256", secret).update(raw).digest("hex");
const valid = expected.length === signature.length &&
  crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(signature));
if (!valid) return NextResponse.json({ error: "Invalid signature" }, { status: 400 });
```

```

switch (event.event) {
  case "payment.captured": {
    const bookingId = await bookingIdForOrder(payment?.order_id);
    await supabase.from("payments").update({ status: "completed", ... });
    await supabase.from("bookings")
      .update({ status: "confirmed" })
      .eq("id", bookingId)
      .eq("status", "pending"); // only promote PENDING bookings
    await sendBookingConfirmation(bookingId); // idempotent -- see below
    break;
  }
  ...
}

```

Listing 25: app/api/webhooks/razorpay/route.ts — signature over the RAW body

Notice `.eq("status", "pending")` on the update. If a booking was already cancelled and re-funded, a late-arriving webhook must not resurrect it.

7.7 Idempotency: sending the confirmation email exactly once

All three paths call `sendBookingConfirmation()`. Without protection, a guest could get three identical emails. The solution is an **atomic claim**:

```

// Atomic claim: only confirmed bookings that haven't been emailed yet.
const { data: booking } = await admin
  .from("bookings")
  .update({ confirmation_email_sent_at: new Date().toISOString() })
  .eq("id", bookingId)
  .eq("status", "confirmed")
  .is("confirmation_email_sent_at", null) // <-- the claim
  .select("id, guest_id, ...")
  .maybeSingle();

if (!booking) return; // already sent, or not confirmed yet

```

Listing 26: lib/emails/bookingConfirmation.ts

The `UPDATE ... WHERE confirmation_email_sent_at IS NULL` is a single atomic statement. Exactly one caller can win it; every other caller gets zero rows back and returns early. This is a compare-and-swap, expressed in SQL.

SAY THIS: *Three different code paths can confirm a booking, so all three might try to email the guest. We make the email idempotent with an atomic claim: a single UPDATE that stamps a timestamp only if it is still null. Postgres guarantees one winner. The losers get zero rows and quietly return.*

7.8 Razorpay Route: splitting the money

If a hotel manager has linked a Razorpay account, the order carries a `transfers` array. Razorpay then automatically settles the manager's share straight to them, and keeps our commission in our account.

```

const { data: payoutAccount } = await supabase.rpc("hotel_payout_account",
  { p_hotel_id: booking.hotel_id });

if (payoutAccount) {
  const basePaise = Math.round(Number(booking.base_price) * 100);
  const managerShare = Math.min(
    Math.round(basePaise * (1 - PLATFORM_COMMISSION_RATE)), // 15% commission
    amountPaise, // never exceed the charge
  );
}

```

```
if (managerShare > 0 && managerShare <= amountPaise) {  
  transfers = [{ account: payoutAccount, amount: managerShare,  
                currency: "INR", notes: { bookingId } }];  
}  
}
```

Listing 27: app/api/payments/razorpay/order/route.ts

The manager's share is computed on **base_price** (the room revenue), not **total_price** — because GST and the platform fee are not theirs. The **Math.min** guard exists because Razorpay rejects an order whose transfers exceed its amount, which can happen when the hotel only requires an advance deposit.

8. Real-Time Messaging

8.1 The data model

- **conversations** — exactly one per (hotel, guest) pair, enforced by `unique (hotel_id, guest_id)`. This is why `getOrCreateConversation()` can safely do “look it up, else insert it”.
- **messages** — body, attachments (jsonb), `sender_role` ('guest' or 'host').

8.2 Unread counts are maintained by a trigger

The browser never increments a counter. Postgres does, on every insert.

```
create or replace function public.on_message_inserted()
returns trigger language plpgsql security definer set search_path = public
as $$
declare v_preview text;
begin
  if new.body is not null and trim(new.body) <> '' then
    v_preview := substring(new.body from 1 for 100);
  else
    v_preview := 'Photo';
  end if;

  update public.conversations
  set last_message_at      = new.created_at,
      last_message_preview = v_preview,
      host_unread = case when new.sender_role = 'guest' then host_unread + 1 else host_unread end,
      guest_unread = case when new.sender_role = 'host' then guest_unread + 1 else guest_unread end
  where id = new.conversation_id;
  return new;
end; $$;

create trigger on_message_inserted_trigger
after insert on public.messages
for each row execute function public.on_message_inserted();
```

Listing 28: supabase/hms_messaging_migration.sql

Because it is a trigger, the counter is correct no matter *how* the message got inserted — from the web app, from a script, from the Supabase dashboard.

8.3 Two kinds of realtime, used for two different things

Supabase Realtime offers several primitives. We use two, and the distinction is worth knowing.

Primitive	What it is	Why we chose it here
<code>postgres_changes</code>	Streams actual database row changes, read off Postgres’s replication log.	Messages must be durable. If it is in the database, it is real.
<code>broadcast</code>	An ephemeral pub/sub message that <i>never touches the database</i> .	The “typing...” indicator. Writing every keystroke to Postgres would be absurd. It is transient by nature, so we send it transiently.

```
const channel = supabase.channel('room-${refinedId}');

channel
  // A. New messages -- durable, from the DB replication stream
  .on("postgres_changes",
    { event: "INSERT", schema: "public", table: "messages",
```



```

    filter: 'conversation_id=eq.${refinedId}' },
    (payload) => {
      const newMsg = payload.new as Message;
      setMessages((prev) => {
        if (prev.some(m => m.id === newMsg.id)) return prev; // de-dupe
        const next = [...prev, newMsg];
        messageCache[activeId] = next;
        return next;
      });
      triggerMarkRead(activeId);
    })
  // B. Typing status -- ephemeral, never stored
  .on("broadcast", { event: "typing" }, (payload) => {
    const { userId, isTyping } = payload.payload;
    if (userId !== currentUserId) setIsOtherPartyTyping(isTyping);
  })
  .subscribe();

return () => { supabase.removeChannel(channel); }; // clean up on unmount

```

Listing 29: components/messaging/useConversationMessages.ts

What is a WebSocket? Normal HTTP is like posting letters: the browser asks, the server answers, the connection closes. A WebSocket is like leaving the phone line open. Either side can speak at any time. That is what lets the server push a message to your screen the instant it is written, with no polling.

Realtime respects RLS. The filter in the subscription is not the security boundary — a user could subscribe to another conversation's id. Supabase Realtime evaluates the messages SELECT policy for the subscribing user before delivering a row. Since that policy calls `can_access_conversation()`, an outsider simply receives nothing.

8.4 Optimistic UI: why sending feels instant

When you press Enter, the message appears *before* the server has heard about it.

```

const tempId = crypto.randomUUID();

// Images render instantly from a local blob URL, before any upload finishes.
const optimisticAttachments = files.map((file) => ({
  url: URL.createObjectURL(file), type: "image",
}));

const optimisticMsg: Message = {
  id: tempId, conversation_id: conversationId, sender_id: currentUserId,
  sender_role: currentUserRole, body, attachments: optimisticAttachments,
  created_at: new Date().toISOString(), read_at: null,
  sending: true, // drives the little "clock" icon
};
setMessages((prev) => [...prev, optimisticMsg]); // <-- appears immediately

try {
  const uploaded = await Promise.all(files.map(uploadToStorage)); // parallel
  const message = await apiSendMessage(conversationId, body, uploaded, tempId);
  setMessages((prev) => prev.map(m => (m.id === tempId ? message : m)));
  optimisticAttachments.forEach((a) => URL.revokeObjectURL(a.url)); // free memory
} catch (err) {
  setMessages((prev) => prev.filter(m => m.id !== tempId)); // roll back
  optimisticAttachments.forEach((a) => URL.revokeObjectURL(a.url));
  throw err;
}

```

Listing 30: components/messaging/useConversationMessages.ts — sendMessage

Two subtleties worth pointing out:

1. We pass `tempId` to the server so the real row gets *the same id*. That way when the realtime INSERT event arrives for our own message, the `prev.some(m => m.id === newMsg.id)` de-duplication check recognises it and does not draw it twice.
2. `URL.revokeObjectURL` is called on both success and failure. `createObjectURL` pins the file in memory until you revoke it — forgetting this is a real memory leak in chat apps.

8.5 Image compression, before upload

```
export async function compressFile(file: File): Promise<File> {
  if (!file.type.startsWith("image/")) return file; // only images

  const canvas = document.createElement("canvas");
  const MAX_DIMENSION = 1200; // plenty for a chat bubble
  // ... scale width/height down, preserving aspect ratio ...
  ctx.drawImage(img, 0, 0, width, height);

  canvas.toBlob(
    (blob) => resolve(new File([blob], name + ".jpg", { type: "image/jpeg" })),
    "image/jpeg",
    0.6, // quality: heavy compression, still clear
  );
}
```

Listing 31: lib/compression.ts

It runs in the browser, so the user’s bandwidth is saved, our Storage bill is saved, and the recipient’s load is instant. A 12 MB phone photo becomes roughly 150 KB.

8.6 The privacy problem, and `shares_conversation_with`

The chat panel shows Call / WhatsApp / Email buttons for the host. Those need the host’s phone number, which lives in `profiles` — a table whose RLS says “you may only read your own row”.

The naive fix (“let guests read manager profiles”) would turn the platform into a phone directory. Instead we wrote a function that answers one narrow question.

```
create or replace function public.shares_conversation_with(p_profile_id uuid)
returns boolean language sql stable
security definer set search_path = public
as $$
  select exists (
    select 1 from public.conversations c
    where c.guest_id = p_profile_id
    and ( c.guest_id = auth.uid()
        or public.can_manage_hotel(c.hotel_id, 'reply_messages')
        or public.get_my_role() = 'admin' )
  );
$$;

create policy "profiles: read conversation guests"
on public.profiles for select
using (public.shares_conversation_with(id));
```

Listing 32: supabase/hms_messaging_names_migration.sql

So contact details become visible *only* once a real conversation exists between those two parties, and only to a party with permission to reply in it.

For the reverse direction (public pages showing reviewer names), we do not expose `profiles` at all. We expose a *view*:

```
-- Exposes ONLY id, full_name, created_at. Never phone, dob, email, or location.
```

```
CREATE OR REPLACE VIEW public.public_profiles AS
  SELECT id, full_name, created_at FROM public.profiles;

GRANT SELECT ON public.public_profiles TO anon, authenticated;
```

Listing 33: supabase/listing_public_migration.sql

SAY THIS: *A guest needs the host's phone number to call them — but only if they are actually talking to that host. So instead of loosening the profiles policy, we wrote a **SECURITY DEFINER** function that checks for a shared conversation, and made that the policy. For public pages we never touch **profiles**; we expose a view containing only name and join date.*

8.7 The contact buttons themselves

No library. Just URL schemes every device understands.

```
<a href="tel:+917736184696">Call</a>
<a href="https://wa.me/917736184696">WhatsApp</a>    <!-- digits only, no + or spaces -->
<a href="mailto:host@example.com">Email</a>
```

9. QR Code Check-In

9.1 What the QR code actually contains

Not a secret. Not a token. A plain URL:

```
import QRCode from "qrcode";

const qrCodeUrl = await QRCode.toDataURL(
  `${origin}/bookings/${booking.id}/check-in`
);
// -> "data:image/png;base64,iVBORwOKGgo..." (an image, inline, no file needed)
```

Listing 34: /page.tsx]components/SuccessClient.tsx / app/bookings/[id]/page.tsx

Why is it safe to put the booking id in a QR code? Because knowing an id grants nothing. To *use* it, the scanner must call `check_in_booking()`, which asks Postgres `can_operate_hotel(booking.id)`. Only that hotel's manager or assigned staff pass. A stranger who photographs your QR code can look at the booking page (RLS blocks them) and check you in (the RPC blocks them). Nothing.

Note also that the QR encodes a *URL*, so a guest who scans their own code with a normal camera app lands on their check-in page. Nice touch, and it means the code degrades gracefully.

Note that `toDataURL` returns a base64 data URI, which we drop straight into an `<img src>`. No file is written, nothing is uploaded, nothing is stored. The QR is regenerated on every page render from the booking id.

9.2 Scanning it (front desk)

```
import { Html5Qrcode } from "html5-qrcode";

useEffect(() => {
  const html5Qrcode = new Html5Qrcode("qr-reader");
  html5Qrcode.start(
    { facingMode: "environment" }, // rear camera on a phone
    { fps: 10, qrbox: (w, h) => { const s = Math.min(w, h) * 0.7;
      return { width: s, height: s }; } },
    (decodedText) => onScan(decodedText),
    () => { /* ignore scan failures -- normal when the QR is out of frame */ },
  );
  return () => { if (html5Qrcode.isScanning) html5Qrcode.stop(); }; // release camera
}, [onScan, onError]);
```

Listing 35: /FrontDesk.tsx]app/manager/manage/[hotelId]/FrontDesk.tsx

The cleanup function is not optional. If you unmount without calling `stop()`, the camera light stays on.

9.3 From scanned text to a check-in

```
onScan={async (decodedText) => {
  // The QR holds a URL; pull the UUID out of it. Fall back to raw text so a
  // plain booking id also scans.
  const match = decodedText.match(/bookings\/([a-f0-9-]{36})/i);
  const bookingId = match ? match[1] : decodedText.trim();

  let matched = bookings.find((b) => b.id.toLowerCase() === bookingId.toLowerCase());

  // The booking may be newer than our cached list -- re-fetch before giving up.
  if (!matched) {
```

```

    const res = await fetch(`/api/manager/manage/${hotel.id}`);
    ...
  }
}

```

Listing 36: /FrontDesk.tsx]app/manager/manage/[hotelId]/FrontDesk.tsx

Then the server action:

```

create or replace function public.check_in_booking(p_booking_id uuid)
returns void language plpgsql security definer set search_path = public
as $$
declare v_b public.bookings;
begin
  select * into v_b from public.bookings where id = p_booking_id;
  if not found then raise exception 'Booking not found'; end if;
  if not public.can_operate_hotel(v_b.hotel_id) then raise exception 'Not allowed'; end if;
  if v_b.status <> 'confirmed' then
    raise exception 'Only confirmed bookings can be checked in'; end if;
  update public.bookings set status = 'checked_in' where id = p_booking_id;
end; $$;

```

Listing 37: supabase/hms_management_migration.sql

Three guards, in order: does it exist, may you touch this hotel, is it in the right state. A pending (unpaid) booking cannot be checked in. A cancelled one cannot. And it cannot be checked in twice, because after the first time its status is no longer 'confirmed'.

SAY THIS: *The QR is just a URL containing the booking id. Security is not in the code — it is in `check_in_booking`, which verifies the scanner is that hotel's manager or staff via `can_operate_hotel`, and that the booking is in `confirmed` state. Scan it twice and the second one is rejected, because the status has moved on.*

10. Maps

10.1 What we use, and what we deliberately did not

Option	Verdict
Google Maps	Rejected. Requires an API key tied to a credit card. The key must be shipped to the browser, where anyone can steal it and run up your bill. Free tier ends.
Mapbox	Rejected. Same key-in-the-browser problem, plus a monthly map-load quota.
Static PNG tiles	Rejected. Not interactive.
MapLibre GL + OpenFreeMap	Chosen. MapLibre is the open-source fork of Mapbox GL (from before it went proprietary). OpenFreeMap serves OpenStreetMap vector tiles with no API key and no rate limit . Renders with WebGL, so panning and zooming are buttery.
Leaflet + CartoDB Voyager	Chosen, as the fallback. If WebGL is unavailable or the vector style fails to load, we transparently swap to Leaflet with raster tiles. The map never disappears.

```
// Rich, free, no-key OpenStreetMap vector tiles (ODbL). "positron" is the clean,
// muted, modern style -- keeps street/place labels but drops the loud OSM colors.
const OPENFREEMAP_STYLE = "https://tiles.openfreemap.org/styles/positron";

// CartoDB Voyager -- the richest of our free raster basemaps. Used as the
// fallback engine (Leaflet) when the vector map fails to load.
const VOYAGER_TILES = "https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png";

/**
 * One interactive map surface. Tries MapLibre + OpenFreeMap (rich vector); if
 * that fails to load, transparently falls back to Leaflet + Voyager so the map
 * never disappears. Both engines share the same pin, popup and zoom controls.
 */
```

Listing 38: components/LocationMap.tsx

10.2 Vector tiles vs raster tiles

Raster tiles are pre-rendered PNG images, 256×256 pixels. Zoom in and they go blurry until the next zoom level downloads.

Vector tiles ship the *geometry* — “this road is a line through these coordinates” — and the browser draws it with WebGL. So labels stay sharp at any zoom, the map can rotate and tilt, and you can restyle it (that is how we get the muted “positron” palette that matches our cream-and-green theme without touching a server).

10.3 Where maps appear

- `app/hotels/[id]/components/LocationSection.tsx` — one pin, the hotel, with a fullscreen mode.
- `app/hotels/ExploreMap.tsx` — every approved hotel as a clickable pin, split-screen with the results grid.
- `app/manager/create-hotel/components/LeafletMap.tsx` — the *picker*. The manager clicks to drop a pin; we store `latitude` and `longitude` on the hotel row. Leaflet is used here because click-to-place needs nothing fancy, and it is smaller.

SAY THIS: *We use MapLibre with OpenFreeMap vector tiles — both fully open source, no API key, no billing. Google Maps and Mapbox both require a key that has to be shipped to the browser, where anyone can lift it. And we kept Leaflet with raster tiles as an automatic fallback, so if WebGL is unavailable the map still renders.*

11. Email Notifications

11.1 Why Brevo, and why HTTP instead of SMTP

```
// Transactional email via Brevo's HTTP API. HTTP (not SMTP) so it works
// reliably on serverless. Server-only: never import this into client code,
// it reads BREVO_API_KEY. All sends are best-effort and never throw -- a failed
// email must never break the action that triggered it.
```

Listing 39: lib/email.ts — the header comment is the answer

Three reasons, all in that comment:

1. **HTTP, not SMTP.** Serverless functions on Vercel are short-lived and often block outbound SMTP ports. An HTTPS `fetch` always works.
2. **Server-only.** It reads `BREVO_API_KEY`. Importing it into a Client Component would leak the key.
3. **Never throws.** `sendEmail` returns `false` on failure. A down mail provider must never fail a paid booking.

And why Brevo over SendGrid/Mailgun/Resend: Brevo's free tier does not require you to own and verify a custom domain, which matters for a student project with no domain to verify.

```
export async function sendEmail(input: SendEmailInput): Promise<boolean> {
  const apiKey = process.env.BREVO_API_KEY;
  if (!apiKey || !from) {
    console.error("[email] BREVO_API_KEY or EMAIL_FROM missing -- skipping send");
    return false;
  }
  try {
    const res = await fetch("https://api.brevo.com/v3/smtp/email", {
      method: "POST",
      headers: { "api-key": apiKey, "content-type": "application/json" },
      body: JSON.stringify({
        sender: { email: from, name: fromName },
        to: [{ email: input.to, name: input.toName }],
        subject: input.subject,
        htmlContent: input.html,
      }),
    });
    if (!res.ok) { console.error("[email] Brevo send failed", res.status); return false; }
    return true;
  } catch (err) { console.error("[email] send error", err); return false; }
}
```

Listing 40: lib/email.ts

11.2 The emails we send

One file each, under `lib/emails/`: `bookingConfirmation`, `bookingCancellation`, `checkInConfirmation`, `checkOutConfirmation`, `refundProcessed`, `staffInvite`, `managerVerification`, `listingDecision`.

11.3 Why email HTML is written with tables and inline styles

If you open `lib/email.ts` you will find `<table role="presentation">` everywhere and every style written inline. This looks like 2003 HTML, and it is deliberate: **email clients strip `<style>` blocks**, and Outlook renders with Microsoft Word's layout engine, which does not understand flexbox or grid. Tables and inline styles are the only things that render identically in Gmail, Outlook, and Apple Mail. The comment in the source says exactly this: *"Inline styles only (email clients strip `<style>`/CSS)."*

The QR code in the confirmation email is fetched from an external QR image service rather than embedded, because many clients block base64 images.

12. Managers, Staff and the Admin Panel

12.1 Staff are real accounts, not sub-users

A staff member has `role = 'staff'` in `profiles` and a row in `hotel_staff` linking them to *one* hotel with a `permissions` `text[]` array (e.g. `{offline_booking, view_occupancy, reply_messages}`).

Every permission check in the database funnels through one of two functions:

- `can_operate_hotel(hotel)` — manager *or any* staff. Used for check-in/check-out.
- `can_manage_hotel(hotel, perm)` — manager *or* staff who hold that specific permission. Used for messaging and room management.

12.2 The invite flow

1. Manager enters an email and ticks permissions.
2. A row is created in `staff_invites` with a random UUID token and `expires_at = now() + 14 days`.
3. Brevo sends the invitation link.
4. The invitee signs up (or logs in) and lands on `app/staff/accept/`.
5. `accept_my_staff_invites()` matches pending invites by the *logged-in user's email*, creates the `hotel_staff` rows, and flips their profile role to `staff`.

Matching on the authenticated email (not just the token) means a stolen link cannot be redeemed by somebody else's account.

12.3 Manager verification

A manager cannot list a hotel until an admin approves them. `proxy.ts` enforces this: any `/manager/*` request from an unapproved manager is redirected to `/manager/waiting`. Statuses are `pending`, `approved`, `rejected`, and `more_info` — the last lets an admin ask for a better document without a hard rejection.

12.4 Every admin action is an audited RPC

```
create or replace function public.admin_review_hotel(
  p_id uuid, p_decision text, p_reason text default null)
returns void language plpgsql security definer set search_path = public
as $$
declare v_uid uuid := auth.uid(); v_h public.hotels;
begin
  if public.get_my_role() <> 'admin' then raise exception 'Not allowed'; end if;
  if p_decision not in ('approved', 'rejected') then raise exception 'Invalid decision'; end if;

  update public.hotels
  set status = p_decision,
      rejection_reason = case when p_decision = 'rejected' then p_reason else null end,
      reviewed_at = now(), reviewed_by = v_uid,
      published_at = case when p_decision = 'approved' then now() else published_at end
  where id = p_id;

  insert into public.admin_audit_log (admin_id, action, target_type, target_id, details)
  values (v_uid, 'hotel_' || p_decision, 'hotel', p_id,
         jsonb_build_object('reason', p_reason, 'name', v_h.name));
end; $$;
```

Listing 41: supabase/hms_admin_migration.sql

The pattern never varies: *check the role, validate the input, do the work, write the audit row* — all in one transaction, so an action can never be performed without being logged.

SAY THIS: *Admin powers do not live in the API layer. They live in **SECURITY DEFINER** functions that begin by asserting `get_my_role() = 'admin'` and end by writing an audit-log row — in the same transaction. So an admin action that is not logged is not possible.*

13. Every Package, and Why It Is There

13.1 Runtime dependencies

Package	What it does	Why this one
<code>next 16.2</code>	The framework: routing, SSR, API routes, caching.	One codebase for frontend and backend. Server Components remove a whole class of API endpoints.
<code>react 19.2</code>	The UI library.	Required by Next.js. v19 gives us Server Components and Actions.
<code>@supabase/supabase-js</code>	The database/auth/storage/realtime client.	The official SDK.
<code>@supabase/ssr</code>	Cookie-aware Supabase clients for server rendering.	Without it, the server cannot read the user's session, so RLS would see every server-side query as anonymous.
<code>razorpay</code>	Server-side SDK for creating orders and fetching payments.	Only used server-side — it holds the secret.
<code>swr</code>	Client data fetching with stale-while-revalidate + preloading.	Tiny (about 4 KB), made by the Next.js team, and its <code>preload()</code> gives us hover-prefetching on the dashboards. Chosen over TanStack Query because we needed caching, not a full query engine.
<code>maplibre-gl</code>	WebGL vector map renderer.	Open-source, no API key.
<code>leaflet</code>	Lightweight raster map.	Fallback engine + the location picker in the create-hotel wizard.
<code>qrcode</code>	Generates the check-in QR as a base64 data URI.	Runs on the server; no image file to store.
<code>html5-qrcode</code>	Reads QR codes from a device camera.	Browser-only; handles camera permissions and decoding.
<code>isomorphic-dompurify</code>	Sanitises HTML.	The rich-text hotel description is manager-supplied HTML. Rendering it raw would be a stored-XSS hole. This version runs in <i>both</i> browser and Node, so we sanitise on write (<code>lib/hotelDraft.ts</code>) <i>and</i> on render (<code>OverviewSection.tsx</code>).
<code>motion</code>	Animation (the library formerly called Framer Motion).	Used in 37 files for page transitions, modals, and the chat. Declarative: <code>animate={...}</code> instead of manual CSS keyframes.
<code>gsap</code>	Timeline animation.	Used once, for the landing-page hero sequence, where we needed precisely choreographed timing.

Package	What it does	Why this one
lucide-react	SVG icon set.	Tree-shakeable; each icon is a React component, so we ship only the icons we use.
@animateicons/react	Animated icons.	Used in the navbar.
canvas-confetti	The confetti burst on booking success.	Pure joy. About 3 KB.

13.2 Development dependencies

Package	Purpose
typescript	Static types. Catches “you forgot this field” before runtime. Everything shared lives in <code>lib/types.ts</code> .
tailwindcss v4 + @tailwindcss/postcss	Utility-first CSS. v4 configures itself in <code>app/globals.css</code> — there is no <code>tailwind.config.js</code> any more.
eslint, eslint-config-next	Linting with the Next.js rule set (catches, e.g., using a hook in a Server Component).
pg	A raw Postgres driver, used only by the local seed script (<code>supabase/seed_hotels.ts</code>). Not shipped.
@types/*	Type definitions for the JS-only libraries above.

13.3 Things we chose *not* to use

Not used	Why not
Redux / Zustand	There is very little global client state. Server Components hold the data; <code>AuthProvider</code> and <code>CurrencyProvider</code> are two small React Contexts. A store would have been ceremony without benefit.
Prisma / Drizzle	An ORM would sit <i>above</i> the database and would tempt us into writing authorisation in TypeScript. We wanted RLS and PL/pgSQL to be the source of truth, so we talk to Postgres through PostgREST and RPC.
Express / Nest	Next.js Route Handlers already are the backend. A second server would need its own deploy, its own auth, its own CORS.
Socket.io	Would require a long-lived server process, which Vercel’s serverless functions are not. Supabase Realtime is a managed WebSocket layer that already knows our RLS policies.
Stripe	Weak UPI support in India. Razorpay is the local standard.

14. Security Summary

If you are asked “how secure is this?”, walk down this list.

1. **Row Level Security on every table.** The database refuses to return rows you may not see. Not the API — the database.
2. **The service-role key is used in exactly one file** (`lib/supabase/admin.ts`) and imported by exactly one caller: the Razorpay webhook, where no user session exists.
3. **Every SECURITY DEFINER function pins `search_path = public`**, closing the classic privilege-escalation path.
4. **Prices are never accepted from the client.** `book_room` reads `rooms.price` itself.
5. **Payment signatures are HMAC-verified with `crypto.timingSafeEqual`**, both for the checkout callback and the webhook (the webhook signs the *raw body*, so we read `request.text()` before parsing).
6. **Manager-supplied HTML is sanitised with `DOMPurify`**, on write and on render.
7. **Overbooking is impossible** by CHECK constraint, independent of application logic.
8. **Emails and payment confirmations are idempotent**, so a retried webhook cannot double-charge or double-send.
9. **Personal data is never exposed by default.** Public pages read `public_profiles` (`id`, `name`, `join date`). Phone numbers unlock only via `shares_conversation_with`.
10. **Every admin action is written to an append-only audit log**, in the same transaction as the action.

15. Frequently Asked Questions

Grouped by the part of the system they attack. Read the question, try to answer out loud, *then* read the answer.

15.1 Architecture and Next.js

Q. Why Next.js and not plain React with an Express backend?

Because we would then be maintaining two applications, two deploys, and a CORS configuration. With Next.js the same project renders the HTML *and* serves the API routes. It also gives us Server Components, which let a page read the database directly and ship zero JavaScript for that work — with plain React, every page would need a loading spinner and an API endpoint.

Q. What is the difference between a Server Component and a Client Component?

A Server Component runs on the server only. Its JavaScript is never sent to the browser, and it can `await` a database query directly. A Client Component (marked "use client") is shipped to the browser and can use state, effects and browser APIs. Our rule: fetch in the Server Component, pass the data as props to a small Client Component for interactivity. That is why the codebase is full of `page.tsx` plus `SomethingClient.tsx` pairs.

Q. How does routing work? Where is the route config?

There is none. The folder structure *is* the routing table. `app/hotels/[id]/page.tsx` serves `/hotels/<id>`. A folder in square brackets is a dynamic segment. A `route.ts` makes the folder an API endpoint rather than a page.

Q. Why is your middleware file called `proxy.ts`?

Next.js 16 renamed the convention from `middleware.ts` to `proxy.ts`. It does the same job — runs before matching requests. Our `matcher` only lists `/dashboard`, `/manager` and `/admin`, so public pages skip it entirely and pay no auth cost.

Q. If middleware protects your routes, what happens if I call the API directly?

Nothing bad. `/api` is deliberately *not* in the `matcher`. Every API route authenticates independently — admin routes call `getAdminContext()`, and every query is subject to RLS regardless. Middleware exists so users get a friendly redirect, not as the security boundary.

Q. What is a Server Action, and is it safe?

A function marked "use server" that a Client Component can import and call like a normal function; Next.js turns the call into a POST behind the scenes. It is *not* inherently safe — it is a public endpoint, and anyone can call it with any arguments. That is why every action in `app/messages/actions.ts` and friends re-verifies the user with `auth.getUser()` and relies on RLS.

Q. Explain your caching strategy.

Three layers. (1) `unstable_cache` caches the expensive `approved-hotels` query result under the tag "hotels" with a one-hour fallback. (2) When an admin approves or a manager edits a hotel, we call `revalidateTag("hotels")` and the cache is dropped immediately. (3) The hotel detail page also sets `export const revalidate = 60`, so its rendered HTML is CDN-cached for a minute. Fast by default, correct the moment data changes.

Q. Why is there a `lib/supabase/public.ts` when `server.ts` exists?

Because `unstable_cache` forbids reading request-scoped data such as cookies — the cached value must not depend on who asked. `server.ts` reads cookies. So for cached queries we use a stateless, cookie-free anon client. RLS still applies; anonymous can only read approved hotels.

Q. What is SWR and why did you use it over TanStack Query?

SWR shows the cached (stale) data instantly, then revalidates in the background. We picked it because it is about 4 KB, comes from the Next.js team, and its `preload()` lets us prefetch a dashboard’s data when the user merely *hovers* its sidebar link — see `managerApiForRoute()` in `lib/swr.ts`. We needed a cache, not a full query engine.

15.2 Supabase and the database

Q. What is Row Level Security, in one sentence?

A rule attached to a table that Postgres silently appends to every query, so a user physically cannot read or write rows the rule excludes — even if they hold a valid API key and bypass our application entirely.

Q. Isn’t it dangerous to put a database key in the browser?

No — that is the design. The anon key gets you *to* the database; RLS decides what you get *out of* it. The dangerous key is the service-role key, which bypasses RLS. That one lives in `lib/supabase/admin.ts`, is never prefixed `NEXT_PUBLIC_`, and is used only by the Razorpay webhook.

Q. Why do you have four Supabase clients?

Browser (manages cookies), server (reads this request’s cookies so RLS knows who you are), public (cookie-free, because caching forbids request-scoped data), and admin (service role, bypasses RLS, webhook only). Four constraints, four clients.

Q. You have a lot of logic in SQL. Why not in TypeScript?

Four reasons. Atomicity: `book_room` reserves inventory, creates the booking and creates the payment in one transaction that rolls back entirely on failure. Concurrency: only the database can take a row lock. Trust: the price is computed from the database’s own numbers, so a tampered client cannot set it. And single-source: the web app and the webhook both confirm payments by calling the same function.

Q. What is SECURITY DEFINER and why is it everywhere?

It makes a function run with the privileges of its creator rather than its caller, so RLS does not apply inside it. We need it for two things: to let a policy read a table without triggering that table’s own policy (avoiding infinite recursion), and to let a guest trigger a controlled write they could not perform directly. Every one of ours pins `search_path = public` so an attacker cannot swap in a malicious schema.

Q. You mentioned RLS recursion. Show me the actual problem.

The `hotels` policy wants to know “is the caller staff on this hotel?”, which reads `hotel_staff`. The `hotel_staff` policy wants to know “does the caller own this hotel?”, which reads `hotels`. Each policy triggers the other, forever. Postgres throws `infinite recursion detected in policy`. We broke the cycle with single-table `SECURITY DEFINER` helpers — `owns_hotel()` and `is_hotel_staff()`

— which read one table each with RLS bypassed. That is what `hms_rls_recursion_fix.sql` is.

Q. How does a user get a profile row?

A trigger, `handle_new_user`, fires after `insert` on `auth.users`. It reads the sign-up metadata we passed to `supabase.auth.signUp()` and inserts the profile — and, if the role is manager, a pending `manager_verifications` row too. Because it is a trigger, a user without a profile is structurally impossible.

Q. How do you make hotel search fast?

Searching is `ILIKE '%goa%'`, and a normal B-tree index cannot serve a leading wildcard — Postgres would scan every row. So we enabled the `pg_trgm` extension and created GIN trigram indexes on `name`, `location`, `city` and `state`. We also created a *partial* index on `(created_at desc) where status = 'approved'`, which satisfies both the filter and the sort of the catalogue query with no sort step.

Q. Why is there a `public_profiles` view?

Because review author names and the host card are public, but phone numbers and dates of birth are not. The view selects only `id`, `full_name` and `created_at`, and *that* is what we grant to `anon`. There is no way to widen it accidentally.

15.3 Booking and concurrency

Q. Two users book the last room at the same instant. What happens?

This is the question I most wanted to be asked. Inside `book_room` we do `SELECT total_count - booked_count ... FOR UPDATE`, which takes a row-level lock on that night's inventory row. The second transaction blocks on that lock, and when it resumes it reads the *updated* count, sees zero available, and raises an exception — which rolls back the whole function, including any nights it had already reserved. And as a final backstop the table has `check (booked_count <= total_count)`, so even a bug in our logic could not overbook the hotel.

Q. Why is availability `total_units - max(booked_count)` and not the sum or the average?

Because a guest needs the room on *every* night of their stay. If the room is full on the middle night, the whole range is unbookable. Taking the maximum booked count across the range gives exactly the number of units free for the entire stay.

Q. Why `i.date < p_check_out` rather than `<=`?

You do not occupy a room on the night of the day you check out. A stay from the 14th to the 16th consumes the nights of the 14th and 15th — two nights, two inventory rows.

Q. Do you create an inventory row for every room for every future date?

No, that would be tens of thousands of rows per hotel. They are created lazily: `insert ... on conflict (room_id, date) do nothing` runs just before we lock the row, so the row exists exactly when it is first needed.

Q. You calculate the price in TypeScript *and* in SQL. Isn't that a bug waiting to happen?

The TypeScript one is a preview so the breakdown updates instantly as you pick dates — and the

source comment says so: “Mirror of the server-side price math in `book_room` (kept in sync for preview).” Its number is never sent to the server. `POST /api/bookings` accepts a room id and dates, nothing else; the database computes the real price from `rooms.price` and `hotels.gst_percent`. If they drifted, the user would see a wrong preview, never a wrong charge.

Q. What is your refund policy and where does it live?

100% at 48 hours or more before check-in, 50% between 24 and 48 hours, nothing under 24 hours. It is computed in `cancel_booking`, which also releases the inventory night by night using `greatest(0, booked_count - num_rooms)` so the count can never go negative.

Q. Why does `cancel_booking` appear in two SQL files?

Migrations are applied in order and each uses `create` or `replace`. The first version (in `booking_schema.sql`) allowed the guest and admins. The later one (in `hms_management_migration.sql`) also allows the hotel’s manager and staff, so the front desk can cancel for a guest who phones the hotel. The later definition wins.

15.4 Payments

Q. Walk me through a payment.

The browser POSTs a booking id to `/api/payments/razorpay/order`. The server reads the booking (RLS proves ownership), converts the total to paise, calls `razorpay.orders.create`, stores the returned order id on the payment row, and returns the order id plus the *publishable* key. The browser opens Razorpay’s checkout modal — card details go straight to Razorpay, never to us. On success Razorpay hands the browser a payment id and a signature, which we POST to `/api/payments/razorpay/verify`. There we recompute `HMAC_SHA256(order_id + "|" + payment_id)` with our secret and compare it in constant time. If it matches, we call `confirm_razorpay_payment` and the booking becomes confirmed.

Q. Why verify on the server? The browser already knows the payment succeeded.

Because the browser is not trustworthy. Anyone can open dev tools and POST `{bookingId, paymentId: "anything"}` to our verify endpoint. The signature is the proof: it is an HMAC keyed with a secret only Razorpay and our server hold. Without the secret you cannot forge it.

Q. What is `timingSafeEqual` and why not just `===`?

`===` short-circuits at the first differing character, so the time it takes leaks how many leading characters were correct. With enough precise measurements an attacker can recover the signature one byte at a time. That is a timing attack. `crypto.timingSafeEqual` always compares every byte.

Q. What if the user’s browser dies right after paying?

Three independent paths confirm a booking, and we need all three. The happy path is the checkout callback. The webhook is Razorpay’s server calling ours directly — it works even with the browser closed, and because there is no user session it uses the service-role client. And the reconcile path runs when the user re-opens the booking: it asks Razorpay `orders.fetchPayments(orderId)` and confirms if a captured payment exists.

Q. If three paths can confirm the booking, does the guest get three emails?

No. The email function performs an atomic claim: a single `UPDATE bookings SET confirmation_email_sent_a`

`= now() WHERE id = $1 AND status = 'confirmed' AND confirmation_email_sent_at IS NULL` that returns the row. Postgres guarantees exactly one caller wins; the others get zero rows and return immediately. It is a compare-and-swap written in SQL.

Q. Why does the webhook read `request.text()` instead of `request.json()`?

Because the signature is computed over the *exact bytes* Razorpay sent. Parsing to JSON and re-serialising would change whitespace and key order, and the HMAC would no longer match. You must hash the raw body.

Q. The webhook updates a booking to confirmed. What stops it resurrecting a cancelled booking?

The update is `.eq("id", bookingId).eq("status", "pending")`. Only a booking still awaiting payment can be promoted. A late webhook for a booking that has since been cancelled matches zero rows.

Q. Why Razorpay and not Stripe?

The platform is India-first. Razorpay has first-class UPI, net banking and Indian card support, prices in INR natively, and has a proper test mode. Stripe's UPI support in India is weak. Razorpay also gave us *Route*, which lets a single payment be split so the hotel's share settles directly to their account while we keep our 15% commission.

Q. How does the commission split work?

If the manager has linked a Razorpay account, the order carries a `transfers` array with `base_price * (1 - 0.15)` in paise. Razorpay settles that to them and keeps the rest with us. Note it is computed on `base_price`, the room revenue — not on `total_price`, because GST and the platform fee are not the manager's money. And we clamp it with `Math.min(managerShare, amountPaise)` because Razorpay rejects an order whose transfers exceed its own amount, which can happen when a hotel only takes an advance deposit.

Q. Are you handling real money?

We are in Razorpay *test mode*, which simulates the full flow — including webhooks and signature verification — with test card numbers, so no real money moves. Switching to production is a matter of changing the API keys; not a line of our code changes.

15.5 Messaging

Q. How is the chat real-time? Are you polling?

No polling. The browser opens a WebSocket to Supabase Realtime and subscribes to `postgres_changes` for INSERT on the `messages` table, filtered to one conversation. Supabase reads Postgres's replication log, so the moment a row is committed it is pushed to every subscriber.

Q. Someone could subscribe to another conversation's id. Isn't that a data leak?

No. The filter is a convenience, not a security control. Supabase Realtime evaluates the `messages` SELECT policy for the subscribing user before delivering any row, and that policy calls `can_access_conversation`. An outsider's subscription simply receives nothing.

Q. Why is the typing indicator not stored in the database?

Because it is worthless one second later. We send it as a Realtime **broadcast**, which is ephemeral

pub/sub that never touches Postgres. Writing every keystroke to disk would be pointless load. Messages, which must be durable, go through `postgres_changes`. Two primitives, two purposes.

Q. How do the unread badges stay correct?

A trigger. `on_message_inserted` fires after every insert and increments `host_unread` or `guest_unread` on the parent conversation depending on `sender_role`, and stores a 100-character preview. The client never increments a counter, so the count is right no matter how the message was inserted.

Q. Why does a sent message appear before the network call finishes?

Optimistic UI. We generate a UUID client-side, append a message with `sending: true`, and render images from a local `URL.createObjectURL` blob so they appear instantly. Then we upload in parallel and insert the real row — *using the same id*. When the realtime event for our own message comes back, the `prev.some(m => m.id === newMsg.id)` check recognises it and does not duplicate. If the upload fails we remove the optimistic message and revoke the blob URLs.

Q. Why revoke the object URLs?

`URL.createObjectURL` pins the file in memory until you revoke it. In a long chat session that is a genuine memory leak, so we revoke on both the success and the failure path.

Q. How do you compress images?

Client-side, before upload, with the Canvas API. We draw the image into a canvas scaled so its longest side is at most 1200 pixels, then call `canvas.toBlob(..., "image/jpeg", 0.6)`. A 12 MB photo becomes about 150 KB. It saves the user's bandwidth, our storage bill, and the recipient's load time.

Q. The guest can see the host's phone number. Doesn't RLS say you can only read your own profile?

Yes, and that is exactly the problem we had to solve. Rather than weakening the policy, we wrote a `SECURITY DEFINER` function `shares_conversation_with(profile_id)` that returns true only if an actual conversation exists between the caller and that profile, and added it as a second `SELECT` policy. So contact details unlock precisely when a real conversation exists — never before. Public pages do not touch `profiles` at all; they read the `public_profiles` view.

15.6 QR check-in

Q. What is inside the QR code?

A URL: `https://.../bookings/<uuid>/check-in`. Generated server-side with the `qrcode` package as a base64 data URI, so nothing is stored or uploaded. Because it is a URL, a guest scanning it with their normal camera app lands on their own check-in page.

Q. So if I photograph someone's QR code I can check in as them?

No. The id is not a credential. To act on it the scanner must call `check_in_booking()`, which asks `can_operate_hotel(booking.hotel_id)` — true only for that hotel's manager or assigned staff. A stranger cannot even read the booking, because RLS blocks the select.

Q. What stops a booking being checked in twice?

`check_in_booking` requires `status = 'confirmed'`. After the first check-in the status is `'checked_in'`, so the second attempt raises “Only confirmed bookings can be checked in”. The state machine is

the guard.

Q. How does the camera scanning work?

The `html5-qrcode` library requests the rear camera (`facingMode: "environment"`) and decodes 10 frames a second. We pull the UUID out of the scanned URL with a regex, falling back to the raw text so a bare booking id also works. The `useEffect` cleanup calls `stop()` — without that, the camera light stays on after you close the dialog.

15.7 Maps, email, and the rest

Q. Why not Google Maps?

It needs an API key that must be shipped to the browser, where anyone can lift it and spend your money, and it needs a credit card on file. MapLibre with OpenFreeMap vector tiles is fully open source, needs no key, and has no rate limit. We kept Leaflet with CartoDB raster tiles as an automatic fallback so the map never disappears if WebGL is unavailable.

Q. What is the difference between vector and raster tiles?

Raster tiles are pre-rendered PNG images — zoom in and they blur. Vector tiles ship geometry and the browser draws it with WebGL, so labels stay crisp at any zoom and we can restyle the map (that muted “positron” palette) entirely client-side.

Q. Why Brevo, and why the HTTP API rather than SMTP?

Serverless functions are short-lived and frequently blocked from outbound SMTP ports, whereas an HTTPS `fetch` always works. Brevo’s free tier also does not require owning a verified domain. And `sendEmail` returns `false` rather than throwing — a mail outage must never fail a paid booking.

Q. Why is the email HTML written with tables and inline styles?

Because email clients strip `<style>` blocks and Outlook renders with Word’s layout engine, which has no flexbox or grid. Tables plus inline styles are the only combination that renders identically in Gmail, Outlook and Apple Mail.

Q. What is the difference between a manager and staff?

A manager owns hotels and must be approved by an admin before they can list anything. Staff are real login accounts, invited by a manager, scoped to *one* hotel through a `hotel_staff` row carrying a permissions array. Every database check goes through `can_operate_hotel()` (manager or any staff) or `can_manage_hotel(hotel, perm)` (manager or staff with that specific permission).

Q. Could a staff member accept an invite meant for someone else?

No. `accept_my_staff_invites()` matches pending invites against the *authenticated user’s email*, not just the token in the link. A forwarded link redeemed by the wrong account matches nothing. Invites also expire after 14 days.

Q. How do you stop an admin from silently doing something?

Every admin power is a `SECURITY DEFINER` RPC that begins with `if get_my_role() <> 'admin' then raise exception` and ends by inserting into `admin_audit_log` — in the same transaction as the change. An unlogged admin action is not possible.

Q. How do you show prices in different currencies?

Prices are stored in INR because that is what Razorpay charges. At display time `lib/currency.ts` detects the visitor's currency from their IP (falling back to the browser locale), fetches live exchange rates, and formats with `Intl.NumberFormat`. Both the rates and the detected locale are cached in `localStorage` — six hours and 24 hours respectively. If anything fails we fall back to showing INR. **Conversion is presentation only**; the charge is always the stored INR amount.

15.8 The hard questions

Q. What is the weakest part of this system?

Three things, honestly. First, `next.config.ts` sets `images.unoptimized = true` and allows `hostname: "**"`, because a manager may paste any image URL. The comment in the file says we should tighten it once all images live in Supabase Storage. Second, the `manager-documents` storage bucket allows anonymous inserts, because the document must be uploaded before the account exists — the schema comment already flags that a production system should move this behind a server action. Third, the older `confirm_payment` RPC (the manual UPI path, from before Razorpay) is trust-based: it takes the guest's word for a UPI reference number.

Q. What would you build next?

Automated tests around `book_room` — a concurrency test that fires N simultaneous bookings at the last room and asserts exactly one succeeds. Then rate limiting on the auth and booking endpoints. Then moving the manager-document upload behind a signed server action.

Q. What was the hardest bug?

The RLS infinite recursion between `hotels` and `hotel_staff`. The error message points at a policy, but the cause is a *cycle* between two policies, so nothing looks wrong in either one individually. The fix — single-table `SECURITY DEFINER` helpers — is the entire content of `supabase/hms_qls_recursion_fix.sql`.

Q. If you had to explain your project's architecture in one sentence?

A Next.js app where the security and the business rules live in Postgres — Row Level Security decides what you can see, and PL/pgSQL functions decide what you can do — so the front end can be careless and the system is still correct.

Q. Why should the database, not the API, enforce security?

Because an API has many entry points and each one is a chance to forget a check. A table has one set of policies. If I add a new endpoint tomorrow and forget to check ownership, RLS still returns nothing. Defence in depth, with the deepest layer doing the real work.

16. Known Limitations and Honest Notes

Being able to name your own weaknesses is worth more than pretending there are none. If asked “what’s wrong with it?”, pick from this list.

1. **No automated tests.** The concurrency logic in `book_room` deserves a test that fires simultaneous bookings at the last room.
2. **Images are unoptimised.** `next.config.ts` disables Next’s image optimiser and allows any HTTPS host, because managers may paste arbitrary image URLs. The mitigation — an Unsplash-specific URL rewriter — lives in `lib/image.ts`, but it only helps for Unsplash.
3. **The `manager-documents` bucket accepts anonymous uploads,** because the document is uploaded before the account exists. The schema comment already flags this: *“For a production system, move this upload behind a server action.”*
4. **The legacy `confirm_payment` RPC is trust-based.** It was the manual-UPI path before Razorpay. It is still granted to authenticated users and should be revoked.
5. **No rate limiting** on auth or booking endpoints.
6. **Currency conversion is display-only** and depends on a third-party FX API. If it is down, everything falls back to INR.
7. **A typo in `supabase/booking_schema.sql` line 23** — `num_rooms int not null default 1` — would make that file fail if run on a fresh database. The live database was created before the typo was introduced, so it has never been noticed. It should be fixed before anyone tries to reproduce the schema from scratch.

That last one is a real bug in the repository. Fixing it before the demo takes ten seconds, and mentioning that you found it while auditing your own schema is a much better story than having someone else find it.

17. Appendix A: Environment Variables

Variable	Exposed?	Purpose
NEXT_PUBLIC_SUPABASE_URL	Browser	Which Supabase project.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Browser	Public key. Safe — RLS guards the data.
SUPABASE_SERVICE_ROLE_KEY	Server only	Bypasses RLS. Webhook only.
NEXT_PUBLIC_SITE_URL	Browser	Auth redirect target.
RAZORPAY_KEY_ID	Server	Razorpay API id.
RAZORPAY_KEY_SECRET	Server only	Signs and verifies payments.
NEXT_PUBLIC_RAZORPAY_KEY_ID	Browser	Publishable key for the checkout modal.
RAZORPAY_WEBHOOK_SECRET	Server only	Verifies the webhook body HMAC.
BREVO_API_KEY	Server only	Sends transactional email.
EMAIL_FROM, EMAIL_FROM_NAME	Server	Sender identity.

The rule. In Next.js, *any* variable prefixed `NEXT_PUBLIC_` is inlined into the JavaScript bundle at build time and is therefore readable by anyone. Everything without that prefix is server-only. Notice that `RAZORPAY_KEY_ID` and `NEXT_PUBLIC_RAZORPAY_KEY_ID` hold the same value — one for server use, one deliberately marked as public. The *secret* never gets the prefix.

18. Appendix B: Glossary

Term	Meaning
App Router	Next.js's routing system where folders are URLs.
Atomic	All-or-nothing. Either every step happens or none does.
CDN	A network of servers worldwide that cache your pages close to users.
Compare-and-swap	Change a value only if it still holds the expected value. Used for our once-only email.
HMAC	A fingerprint of a message made with a shared secret. Proves who sent it and that it was not altered.
Idempotent	Doing it twice has the same effect as doing it once.
ISR	Incremental Static Regeneration — a page is pre-rendered and refreshed on a timer.
JWT	JSON Web Token. A signed blob proving who you are.
Optimistic UI	Showing the result before the server confirms it, and rolling back if it fails.
Paise	1/100 of a rupee. Razorpay works in integer paise to avoid floating-point money bugs.
PostgREST	The service that turns Postgres tables into a REST API.
Race condition	Two operations interleaving in a way that corrupts data.

Term	Meaning
Row lock	<code>SELECT ... FOR UPDATE</code> . Makes other transactions wait for that row.
RLS	Row Level Security. Per-row access rules enforced by Postgres.
RPC	Remote Procedure Call. Here: calling a Postgres function from the app.
RSC	React Server Component. Runs on the server, ships no JavaScript.
SECURITY DE-FINER	A Postgres function that runs with its creator's privileges, bypassing RLS internally.
Server Action	A server function a Client Component can call directly.
SWR	Stale-While-Revalidate. Show cached data now, refresh in background.
Timing attack	Learning a secret by measuring how long a comparison takes.
Trigram index	An index over every 3-character substring, making <code>ILIKE '%x%'</code> fast.
WebSocket	A connection that stays open so the server can push data.

BookNest

Built by Team Neuron

Good luck tomorrow. You know this system — now you can prove it.